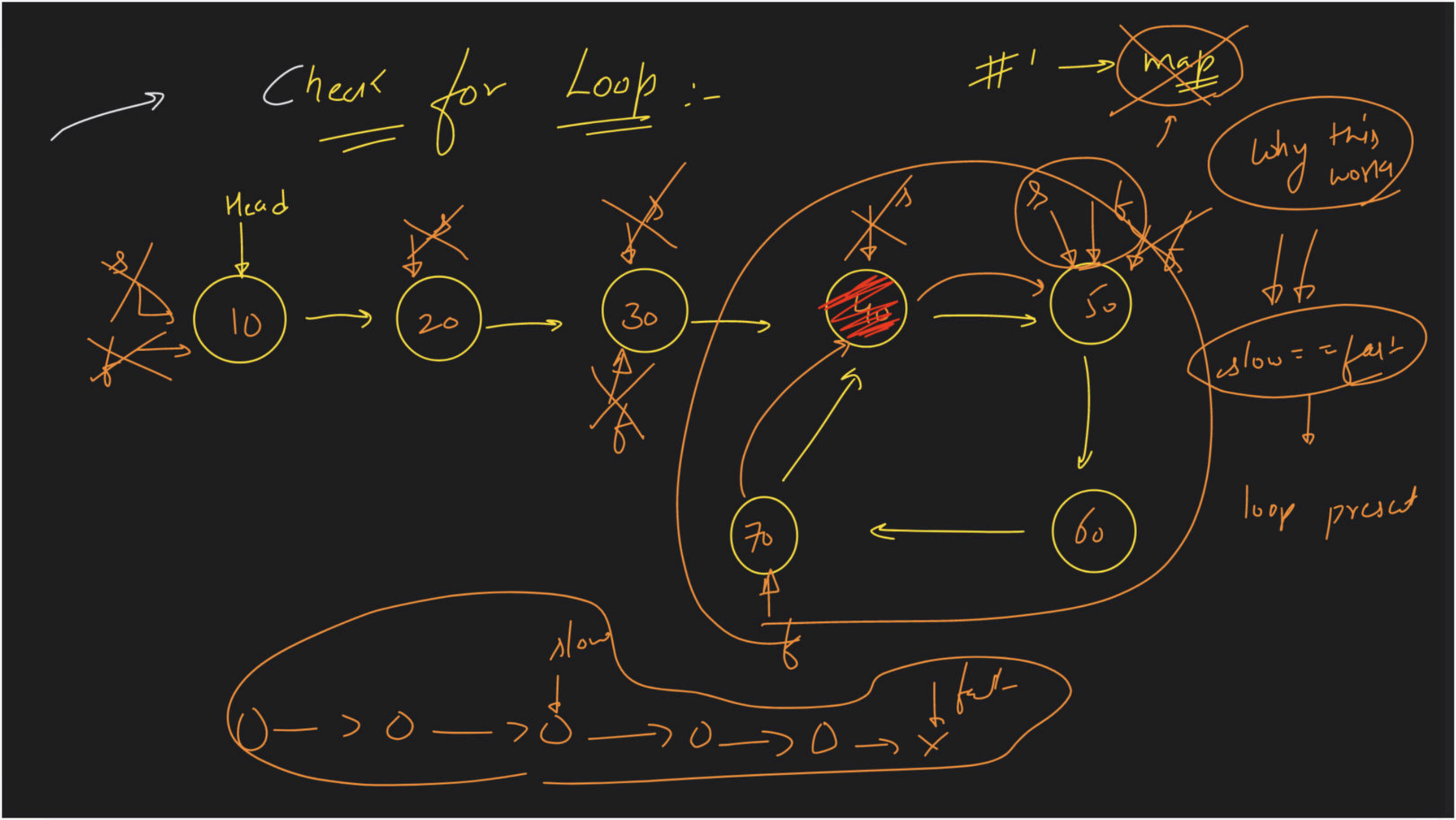
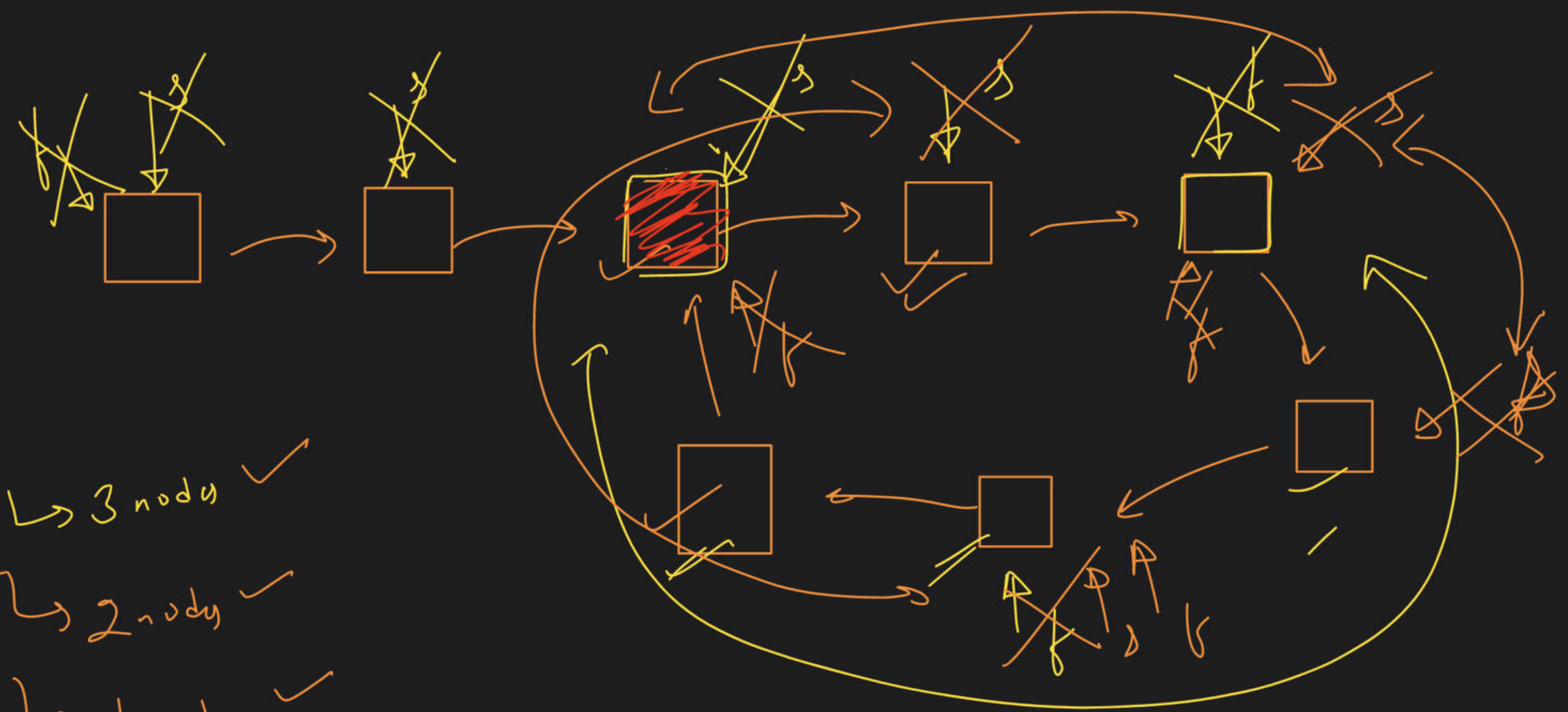
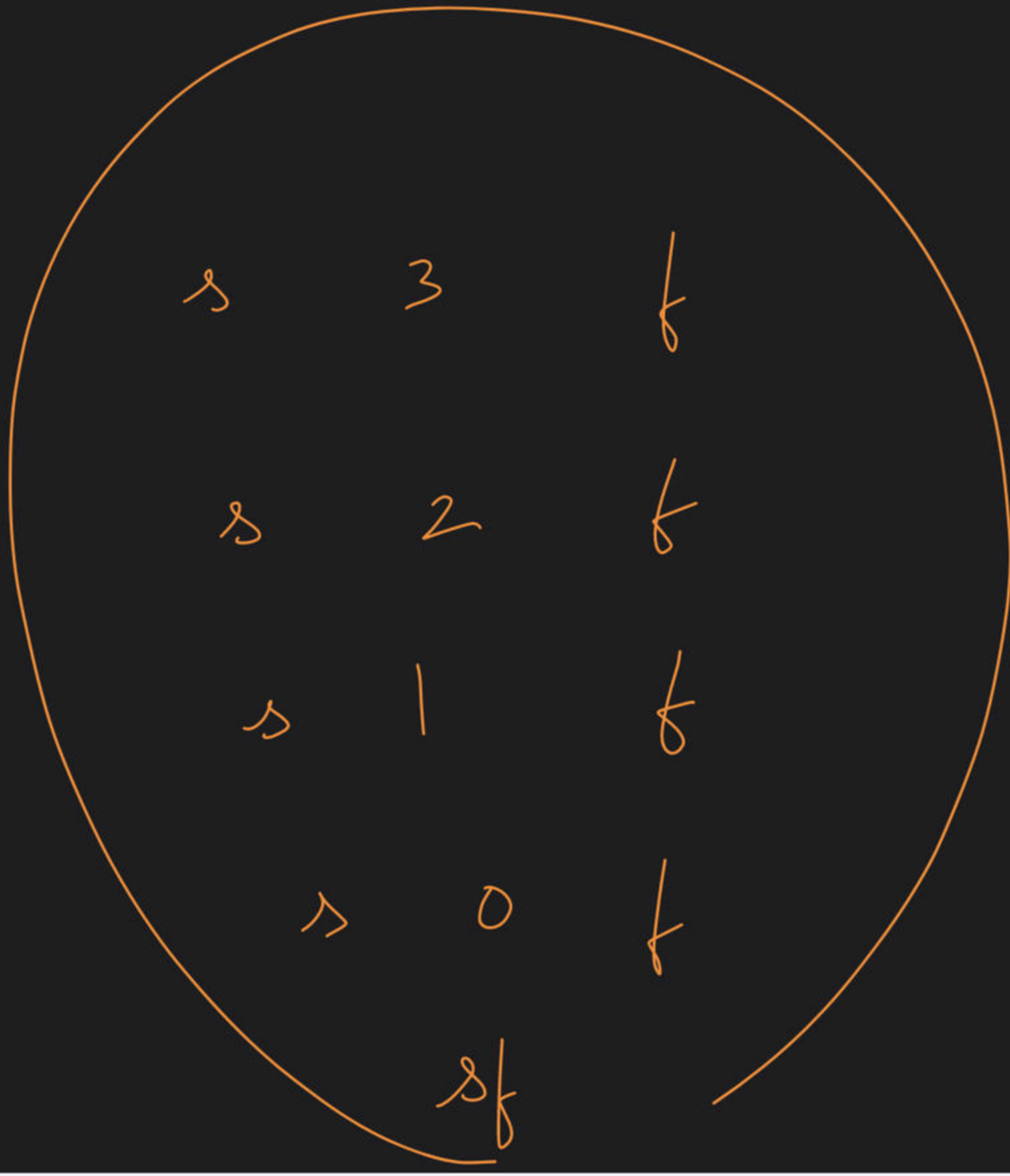
Linked List - Class 4

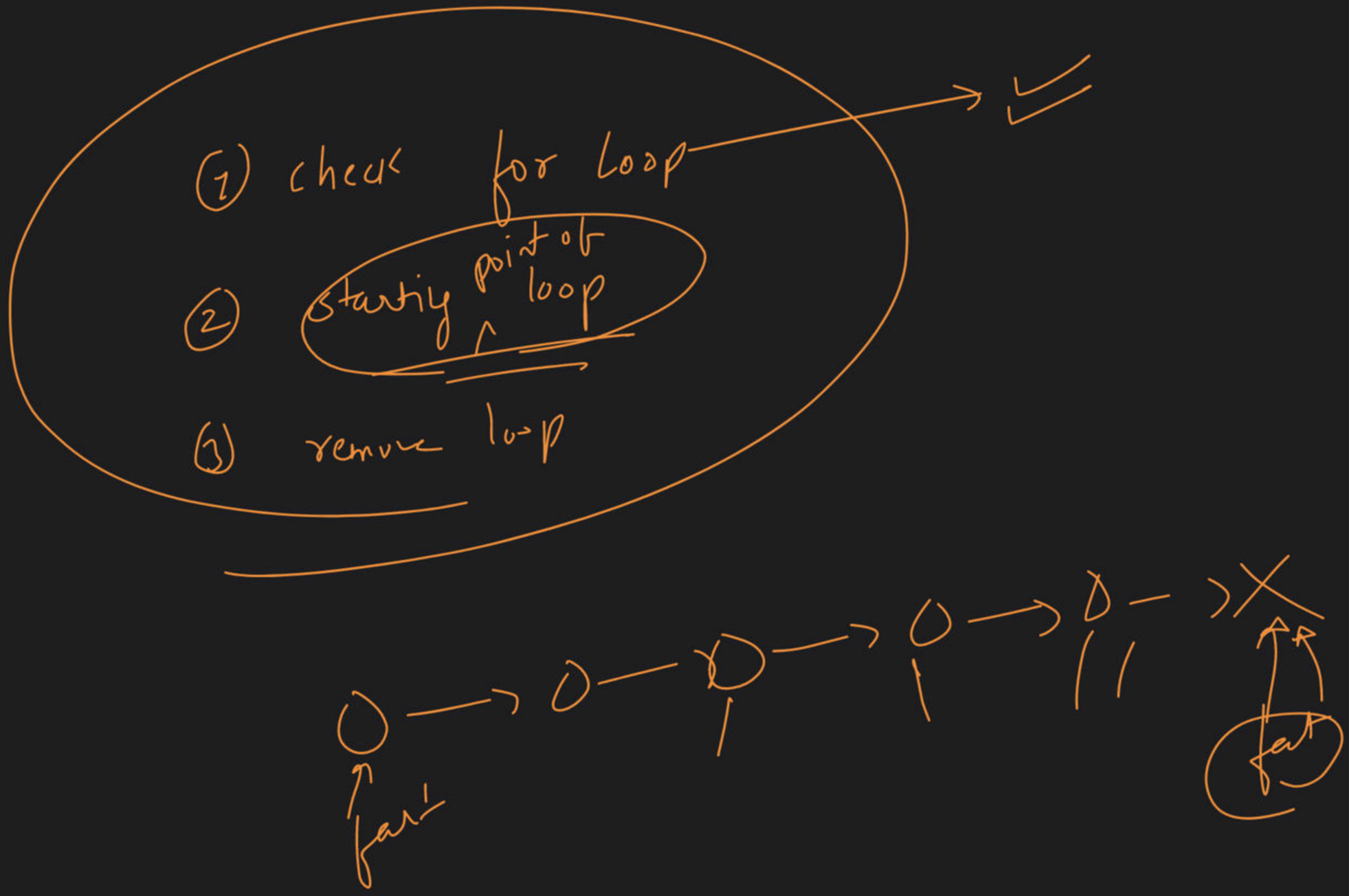
Special class

Love Babbar • Nov 6, 2023


Check for Loop :-
#1 → map
Why this work
Head
10
20
30
40
50
60
70
slow == fast
loop present
slow
fast

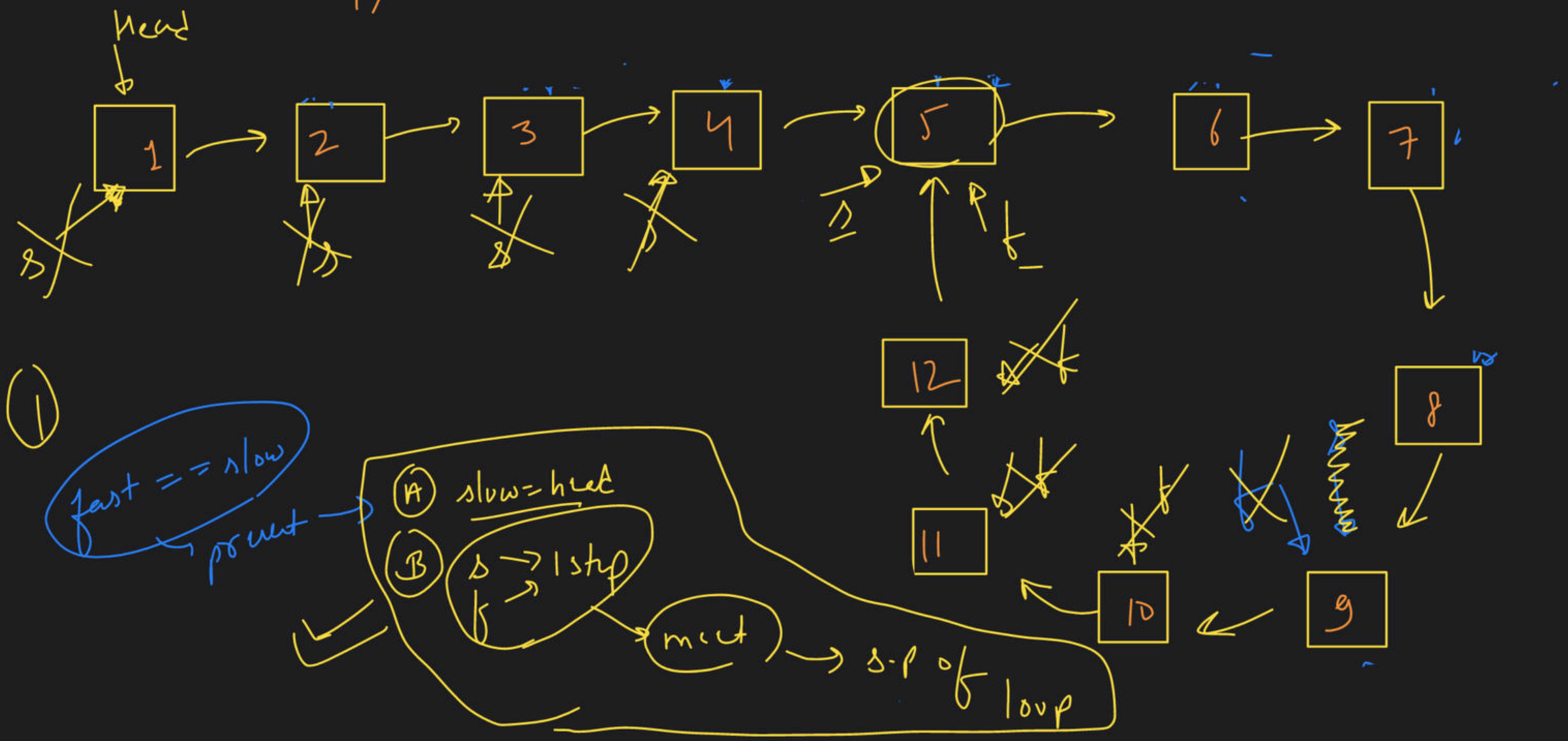

↳ 3 nodes
↳ 2 nodes
↳ 1 nodes
↳ 0 node

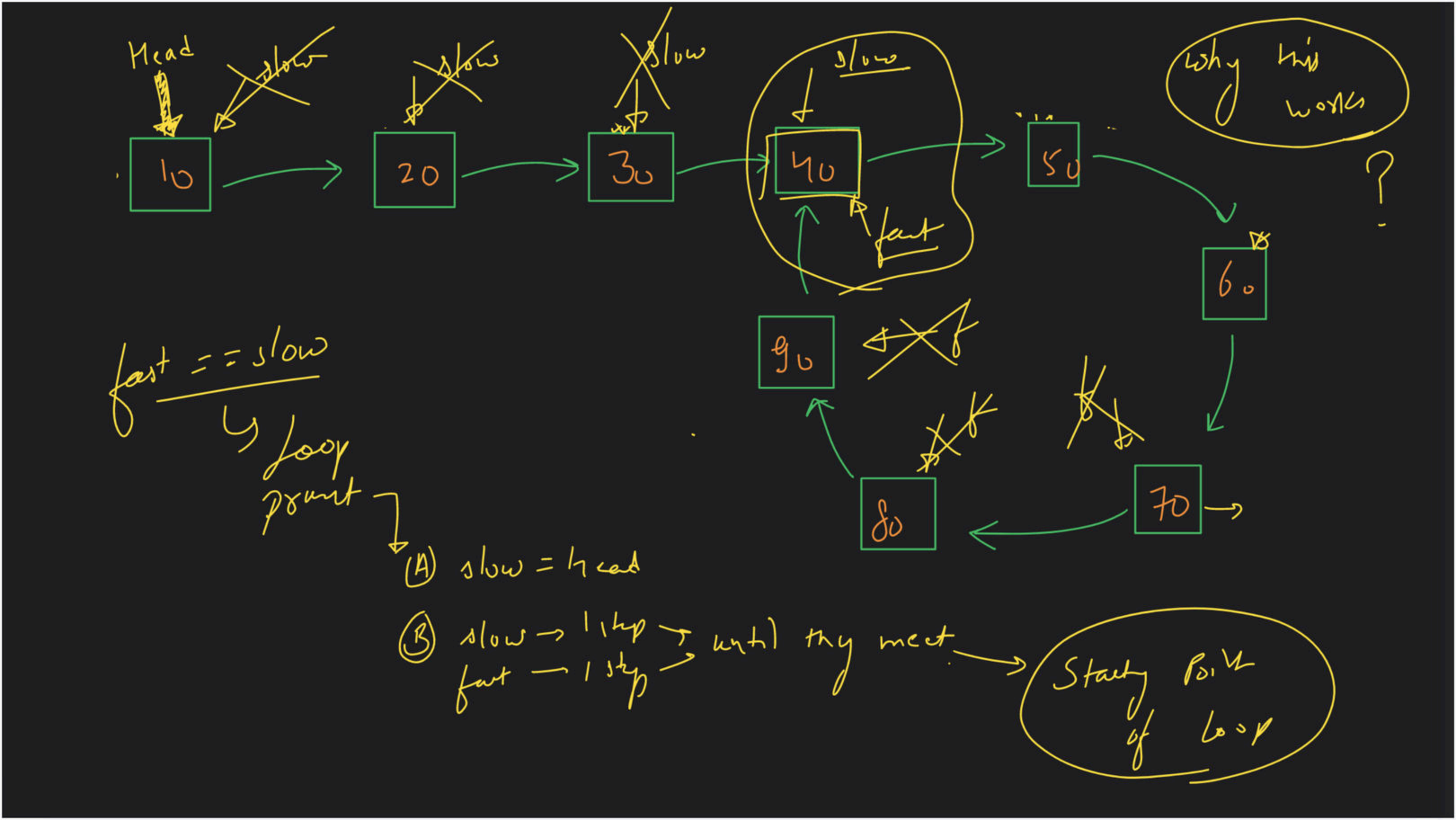

১ ৩ ৳
১ ২ ৳
১ ১ ৳
১ ০ ৳
১৳

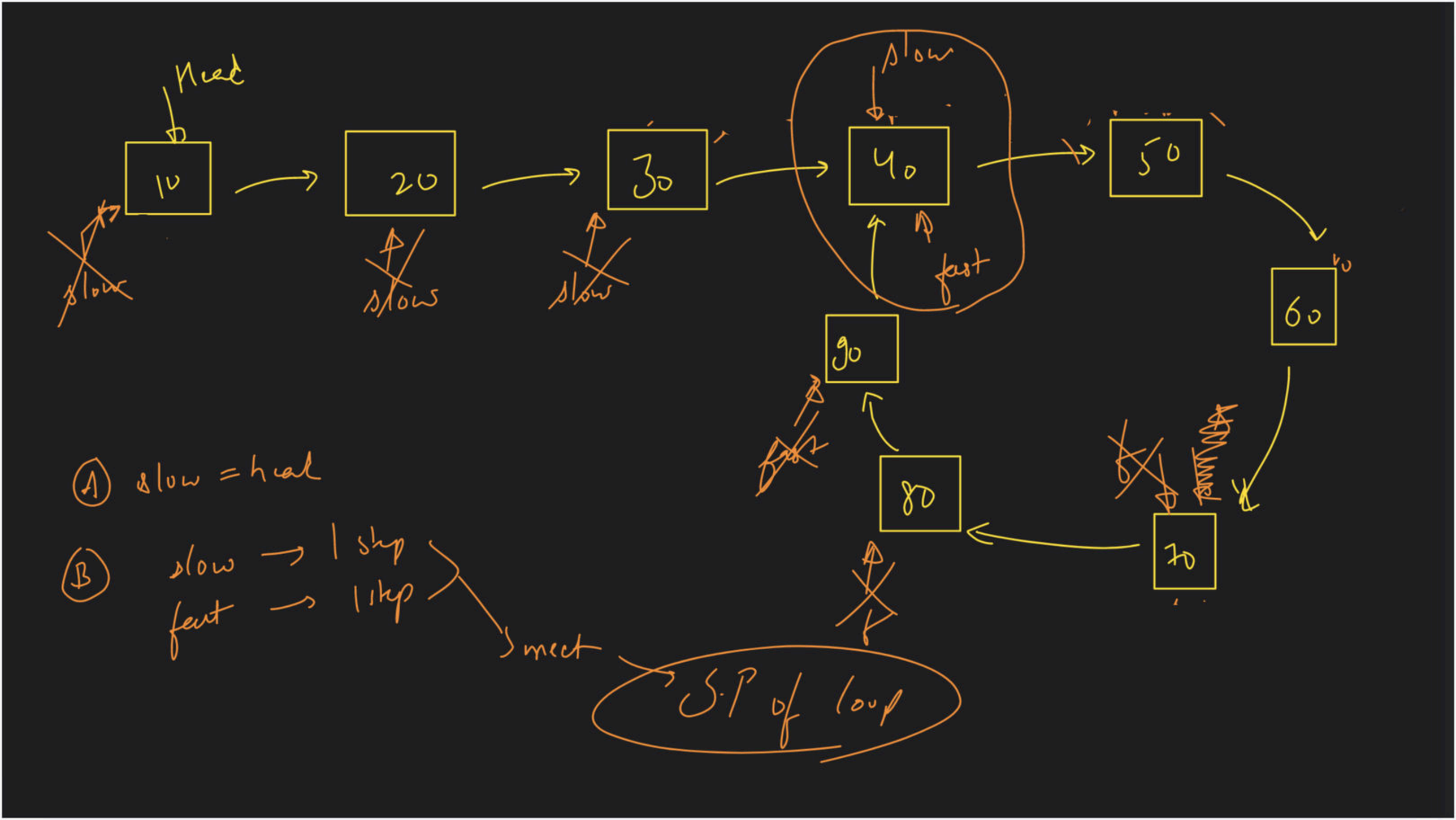

(1) check for loop
(2) starting point of loop
(3) remove loop
fast
fast

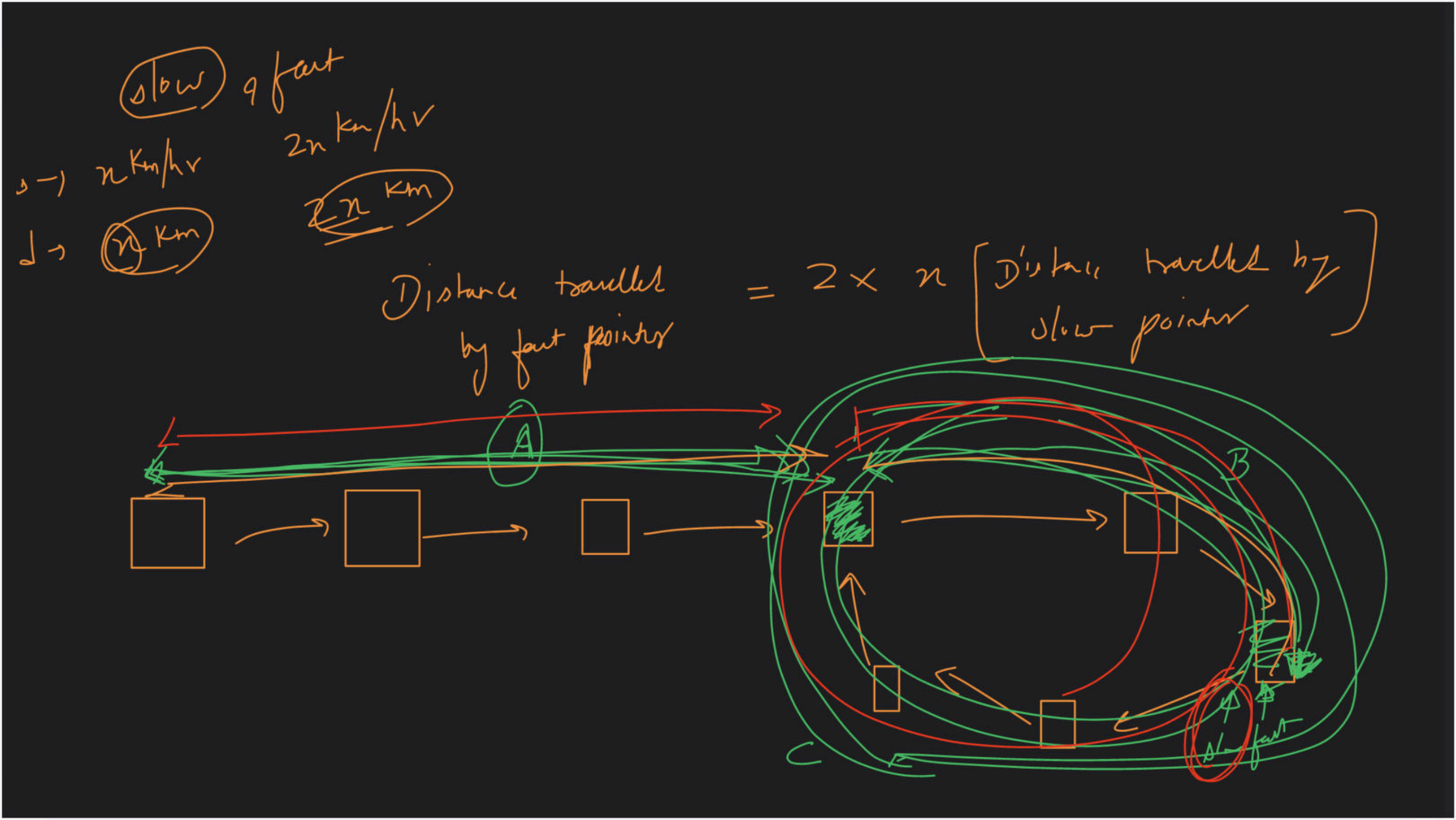
→ Starting Point of Loop :-

S.P of loop → ?




Head
slow
10
20
slow
30
slow
slow
40
fast
50
why this works ?
60
70
80
90
fast == slow
↳ loop present
(A) slow = head
(B) slow → 1 step
fast → 1 step
until they meet
Starting Point of Loop


Head
10
20
30
40
50
60
70
80
90
slow
slow
slow
slow
fast
(A) slow = head
(B) slow → 1 skip
fast → 2 skip
meet
S.P of loop


slow
fast
x Km/hr
2x Km/hr
x Km
x Km
Distance travelled by fast pointer = 2 × x [Distance travelled by slow pointer
A
B
C
slow
fast

Distance travelled by fast pointer = 2 × Distance travelled by slow pointer

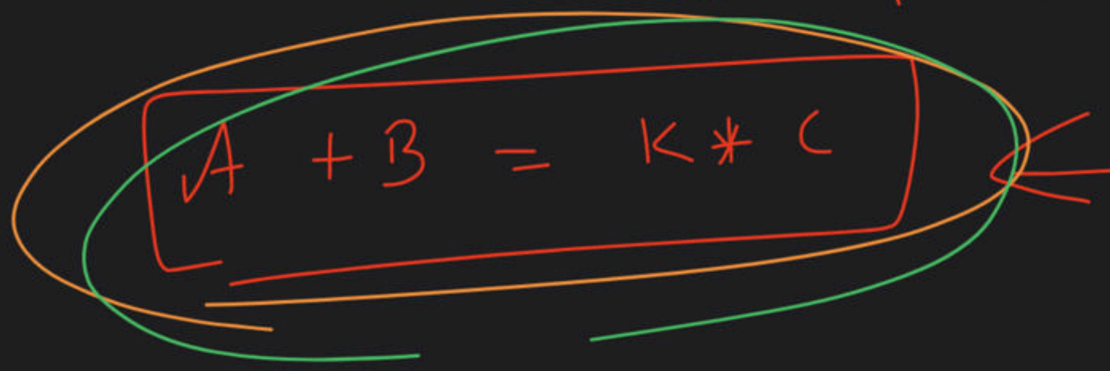
$$A + K_1 \times C + B = 2 * [A + K_2 C + B]$$

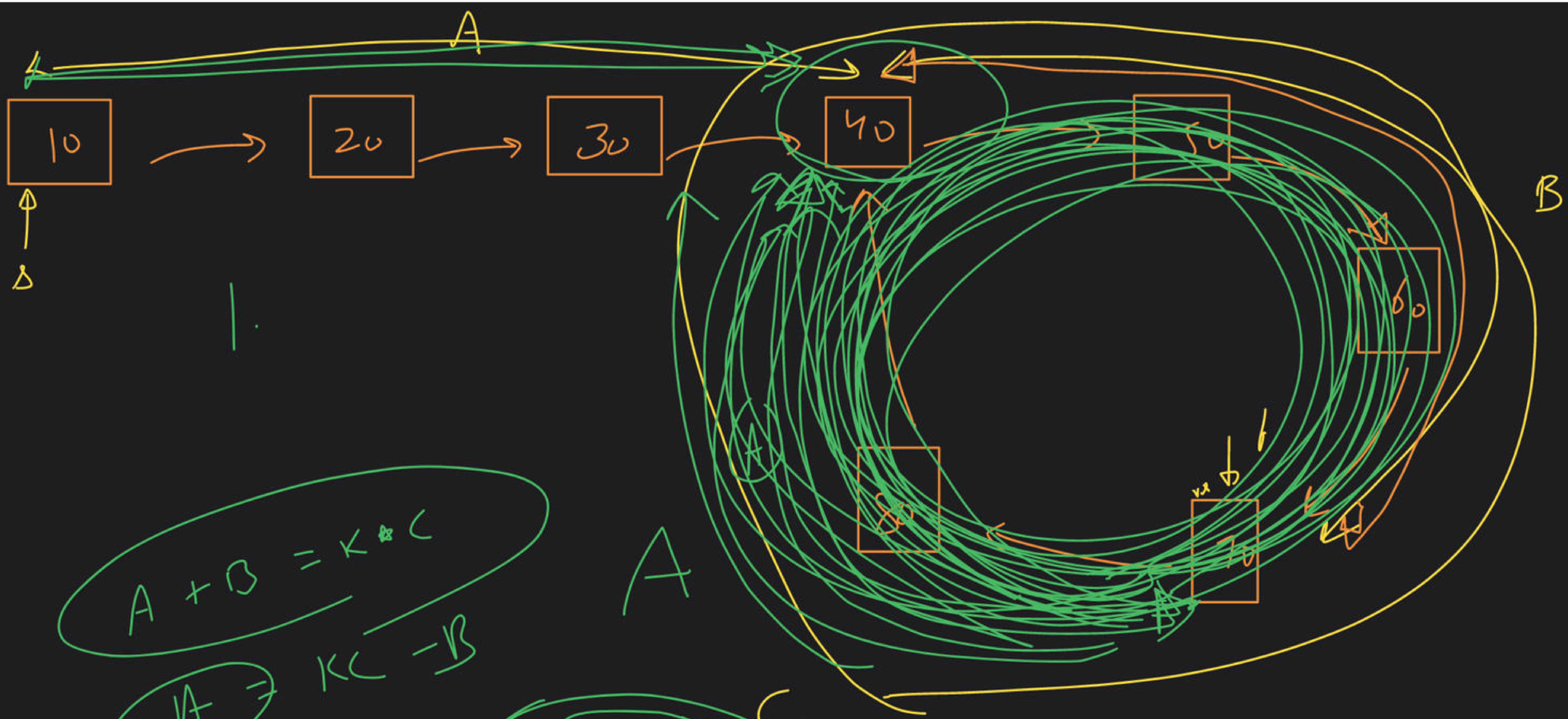
$$A + K_1 C + B = 2A + 2K_2 C + 2B$$

$$(K_1 - K_2) C = A + B$$

let $K_1 - K_2 = K$

$$A + B = K * C$$

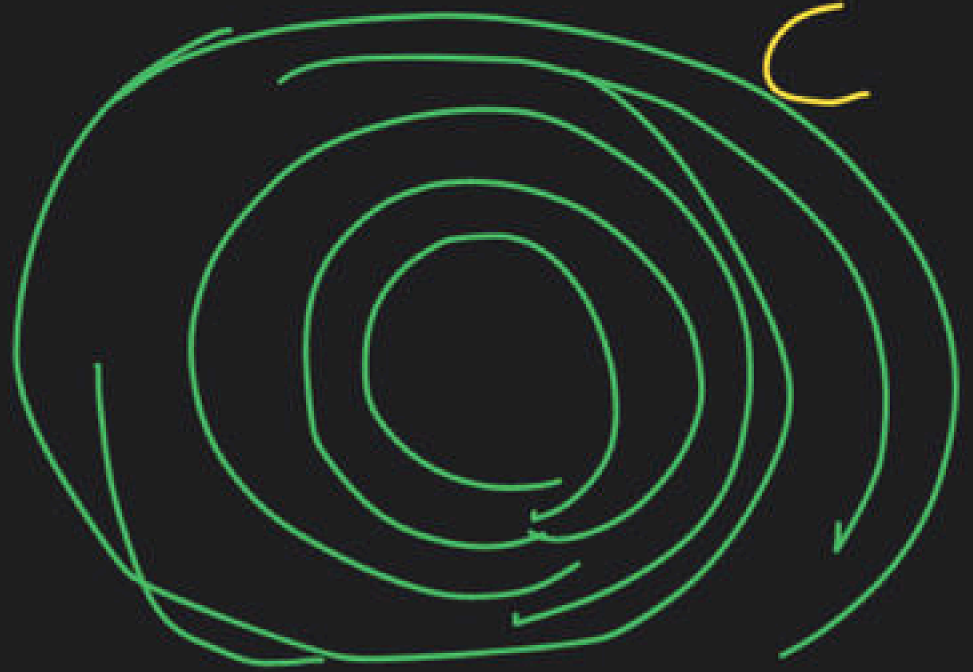




$A + B = K * C$

$A = KC - B$

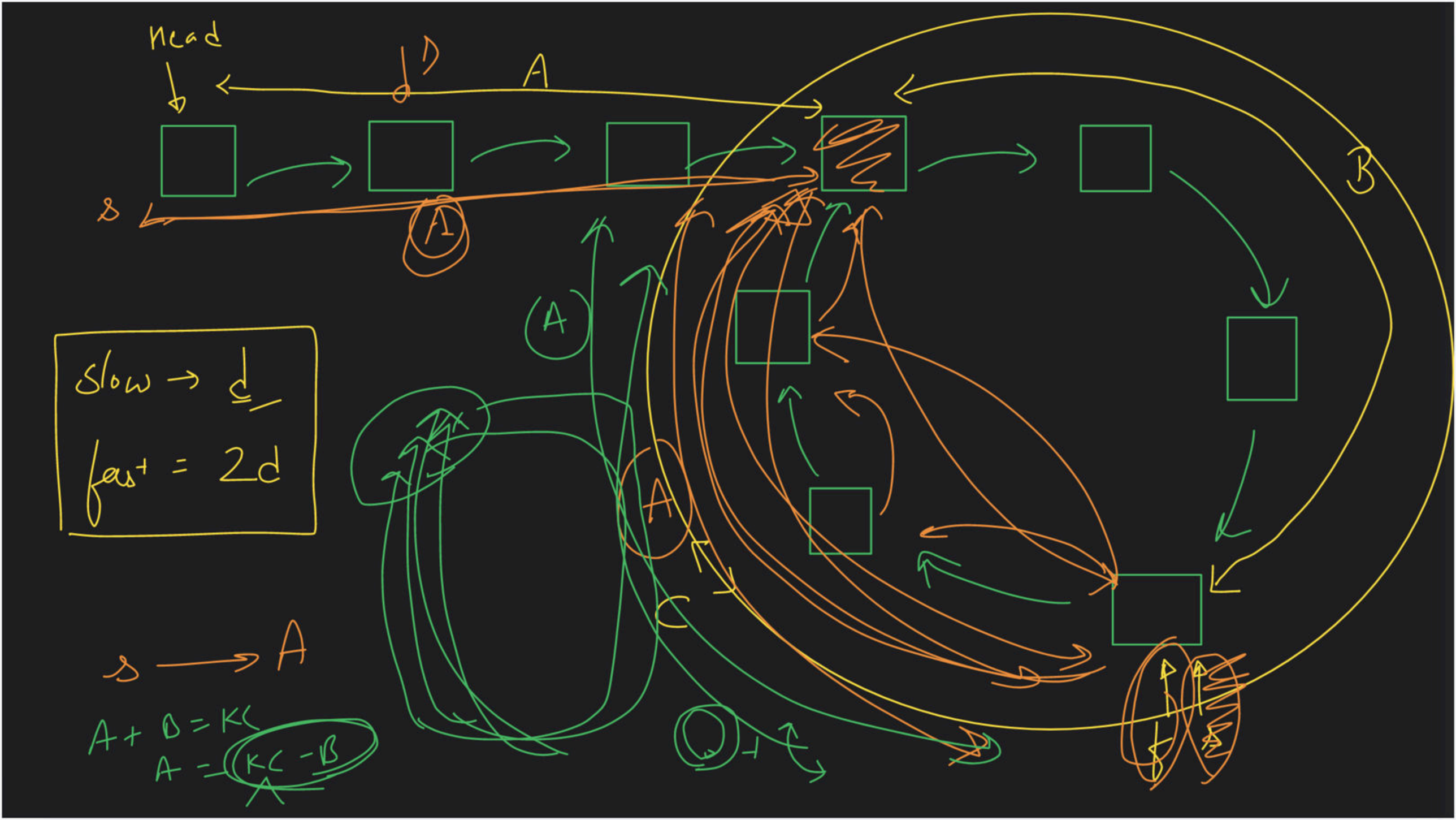
A



$xC + (C - B)$

$(x+1)C - B$

$C - B$


head
d
A
B
A
A
A
slow → d
fast = 2d
s → A
A + B = KC
A = (KC - B)
A

fast = 2d

fast = 2 * slow

Ikram

distance travelled by fast = 2 * distance travelled by slow

$nC + C - B$

$(n+1)C - B$

$KC - B$

$A + K_1 * C + B = 2 * [A + K_2 * C + B]$

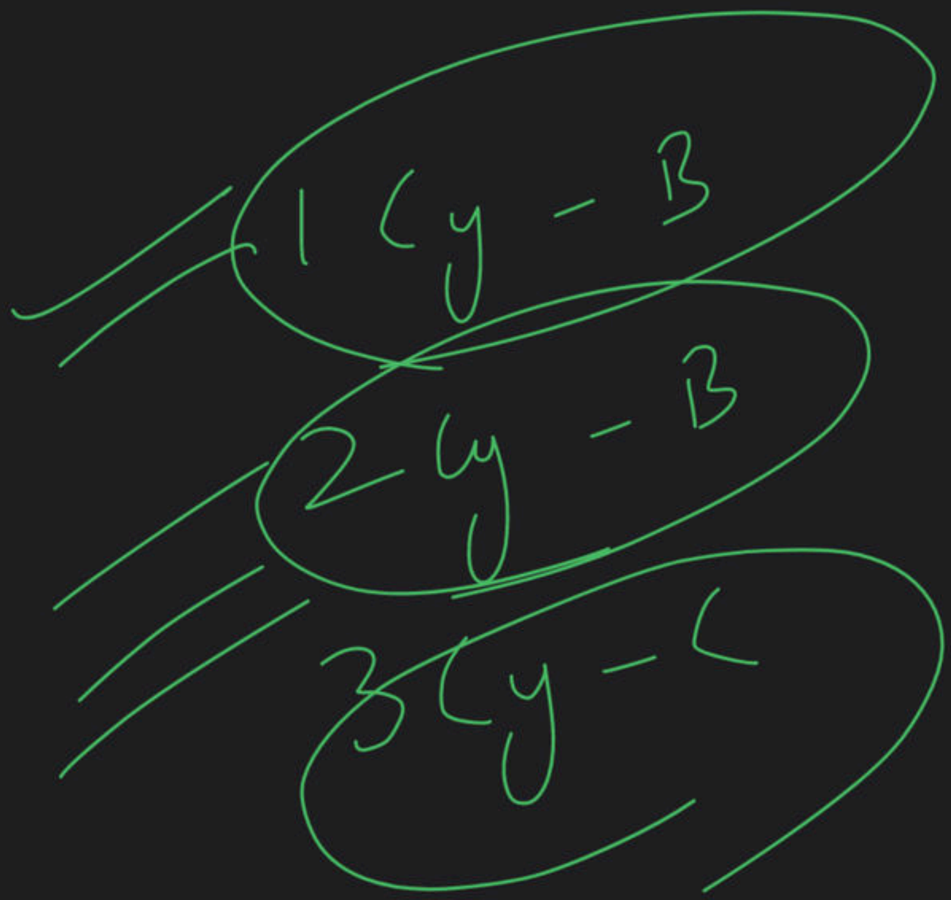
$A + B + K_1 C = 2A + 2B + 2K_2 C$

$K_1 C - K_2 C = 2A + 2B - A - B$

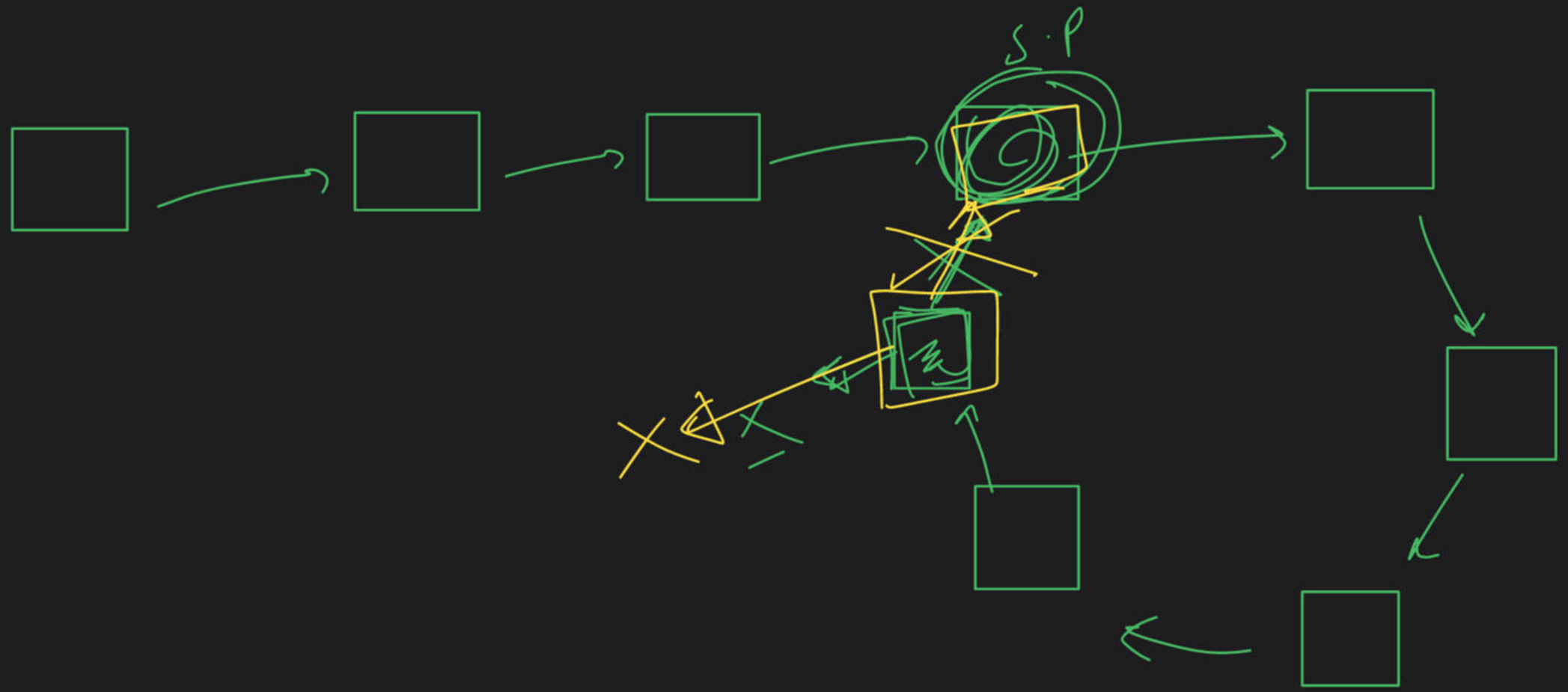
$(K_1 - K_2)C = A + B$

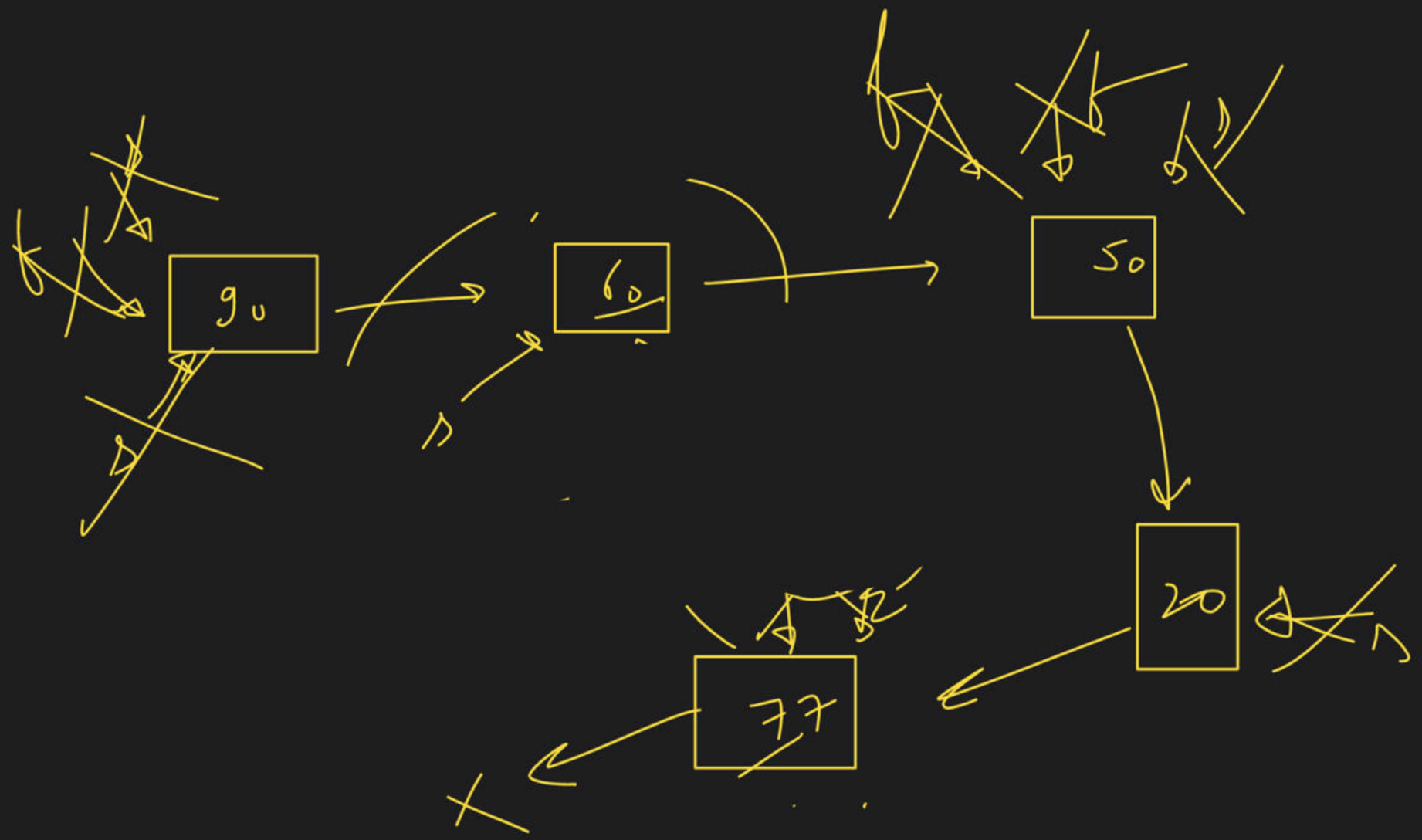
$K_1 - K_2 = K$

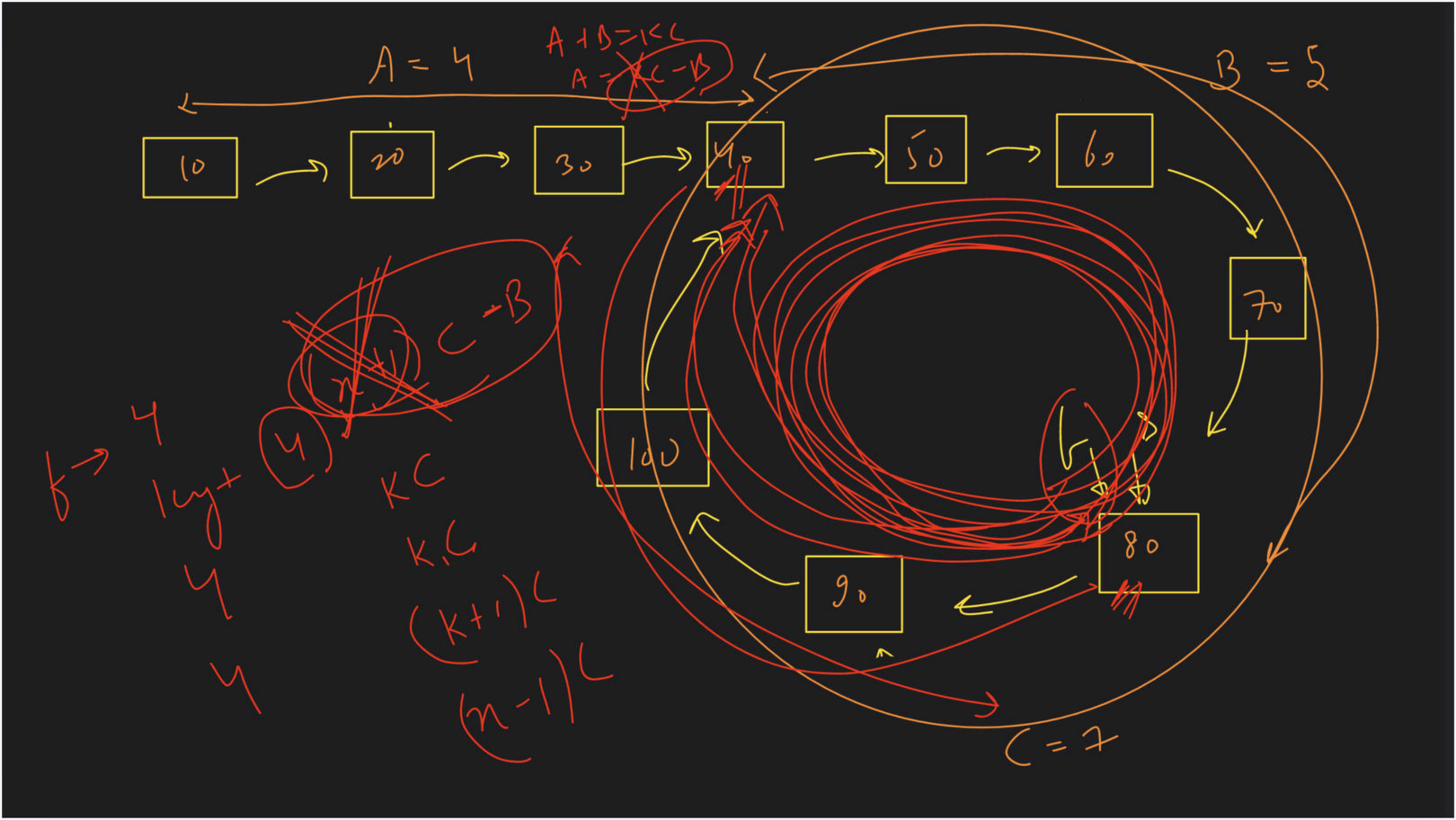
$A + B = KC$


1 (y - B
2 y - B
3 (y - C

③ Remove a Loop

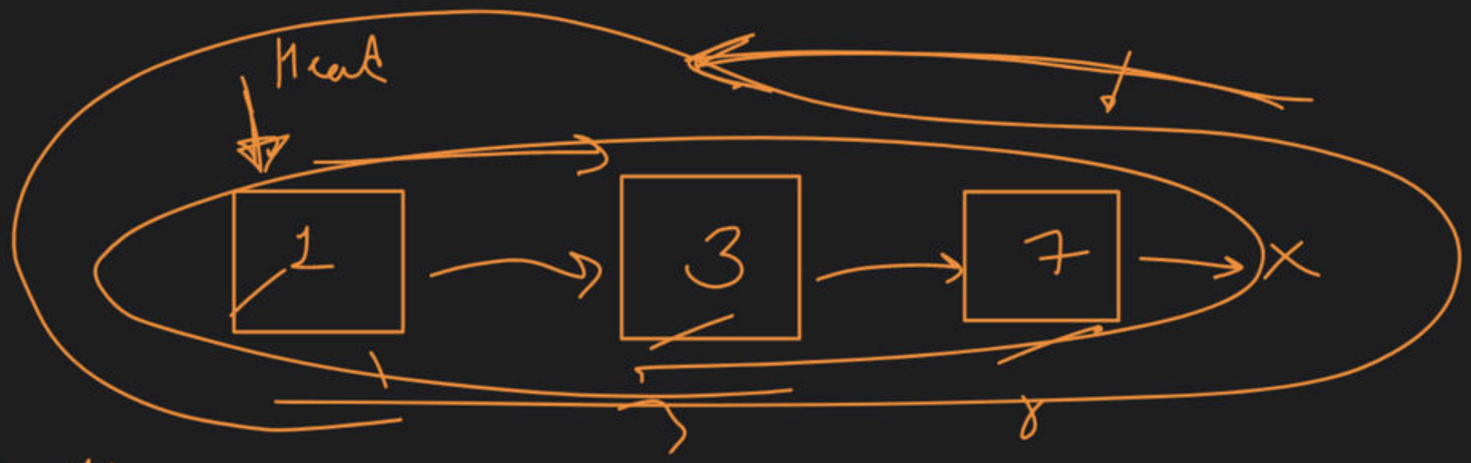



90
60
50
20
77


A = 4
A + B = K C
A = K C - B
B = 5
10
20
30
40
50
60
70
80
90
100
C - B
K C
K . C
(K+1)C
(n-1)C
C = 7

2 min

Add 1 to a LinKed List



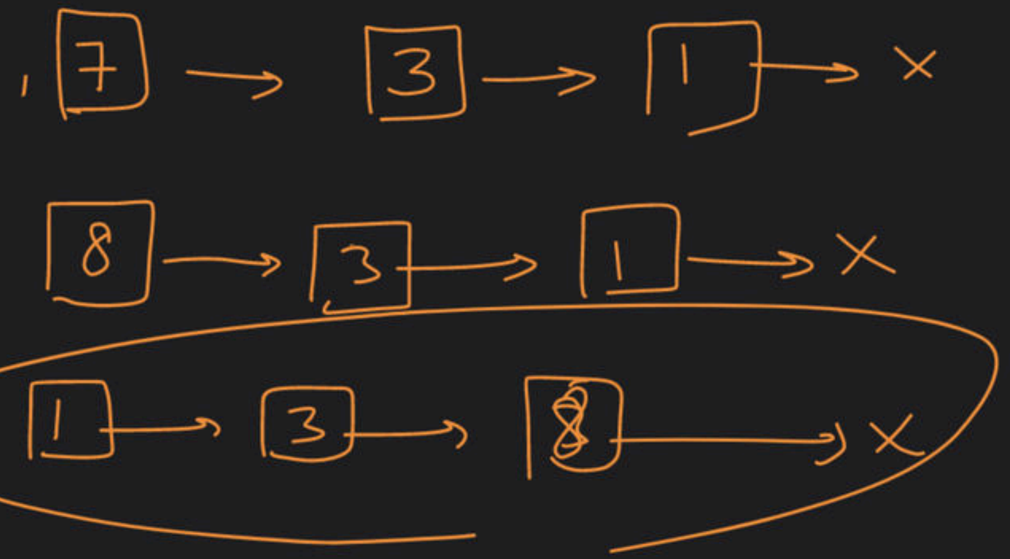
reverse

A

+

B

C

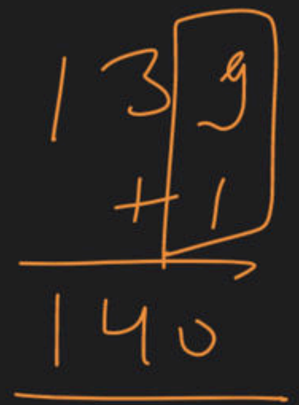


A) rev
B) add
C) rev

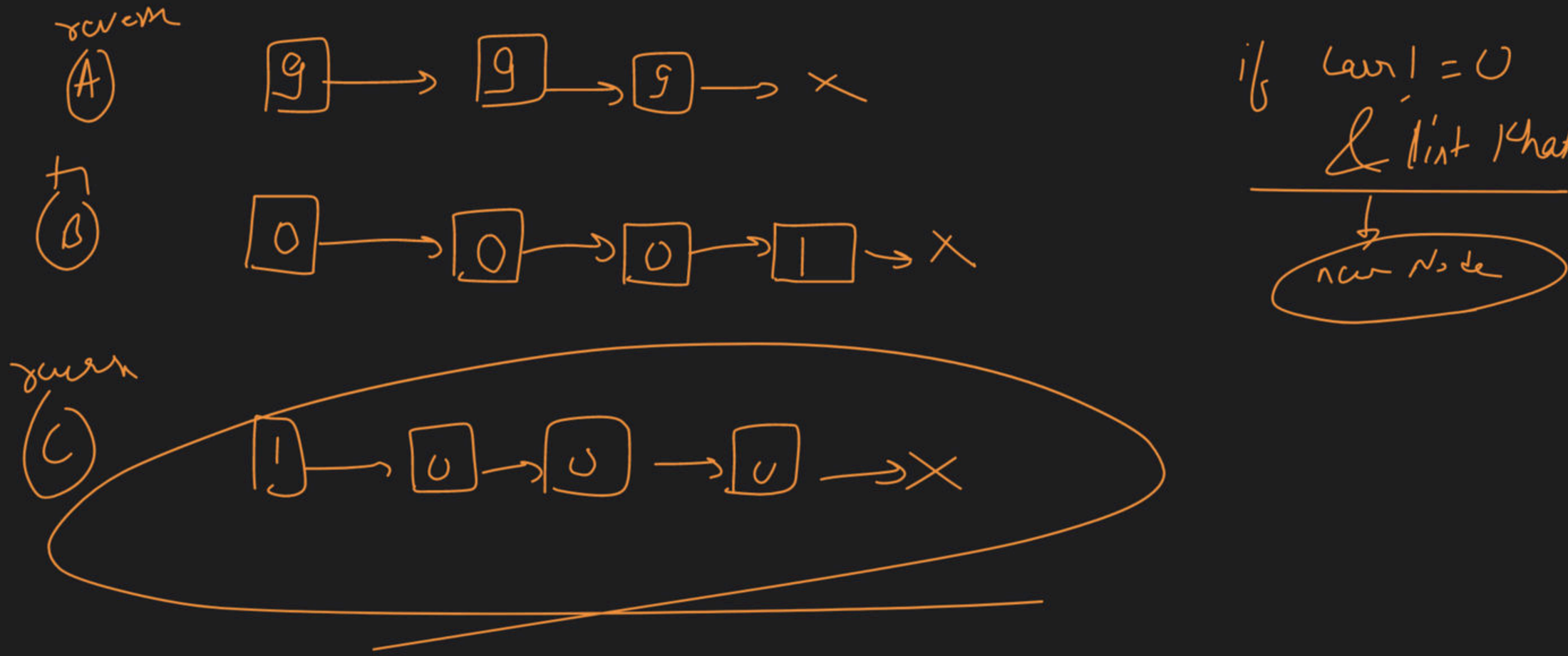
carry -> 0 -> (Head

137

134

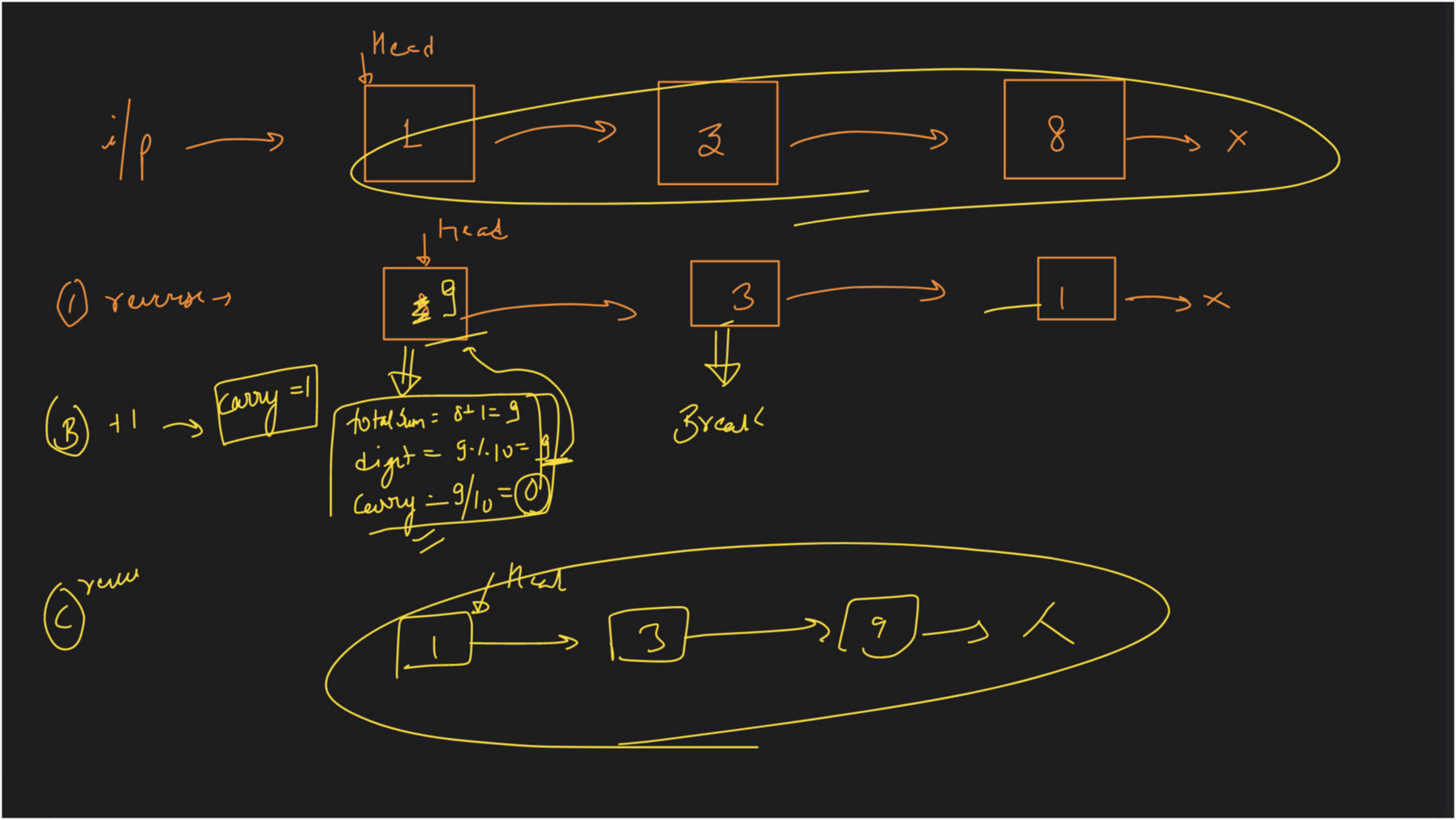


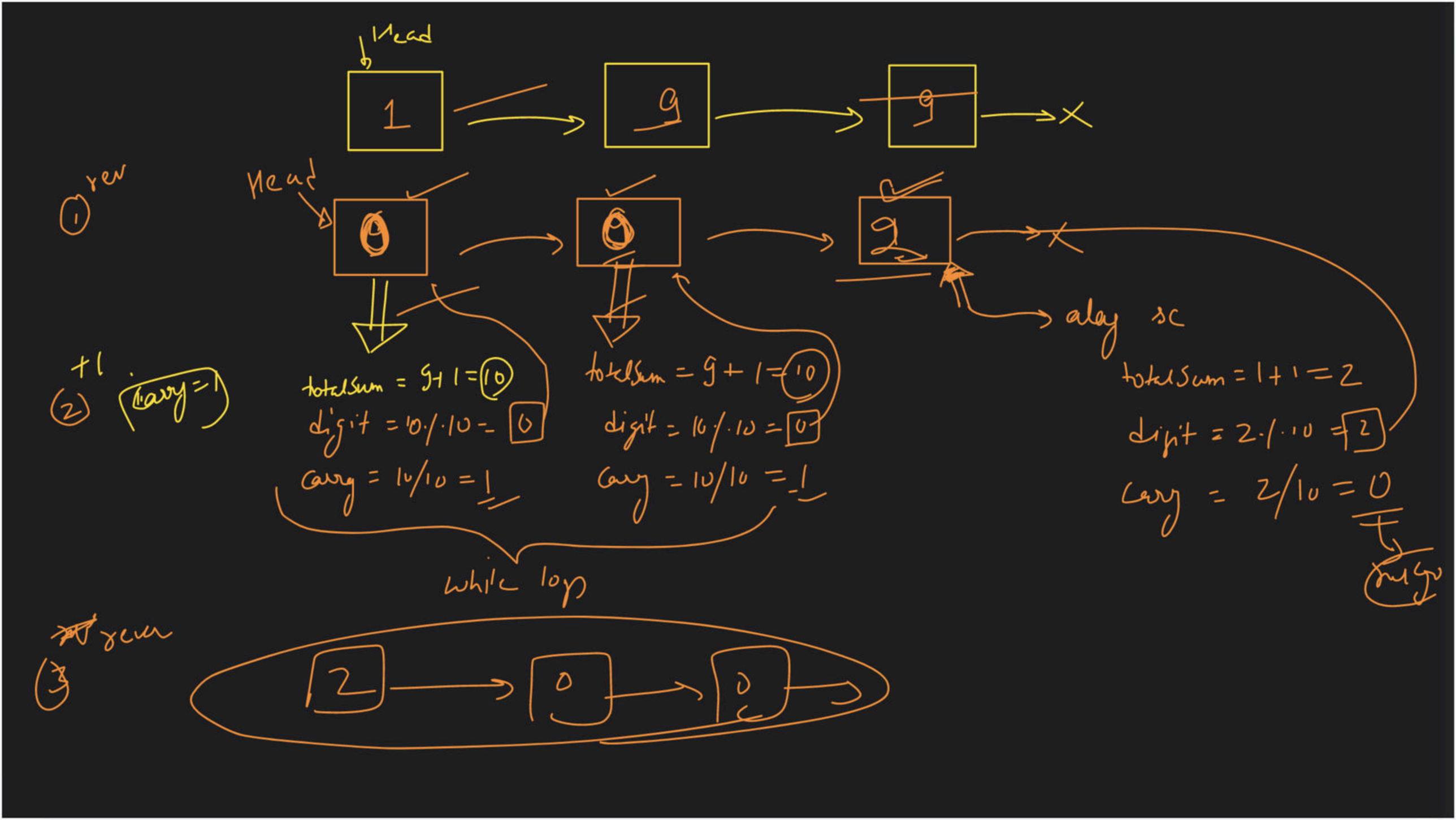
140

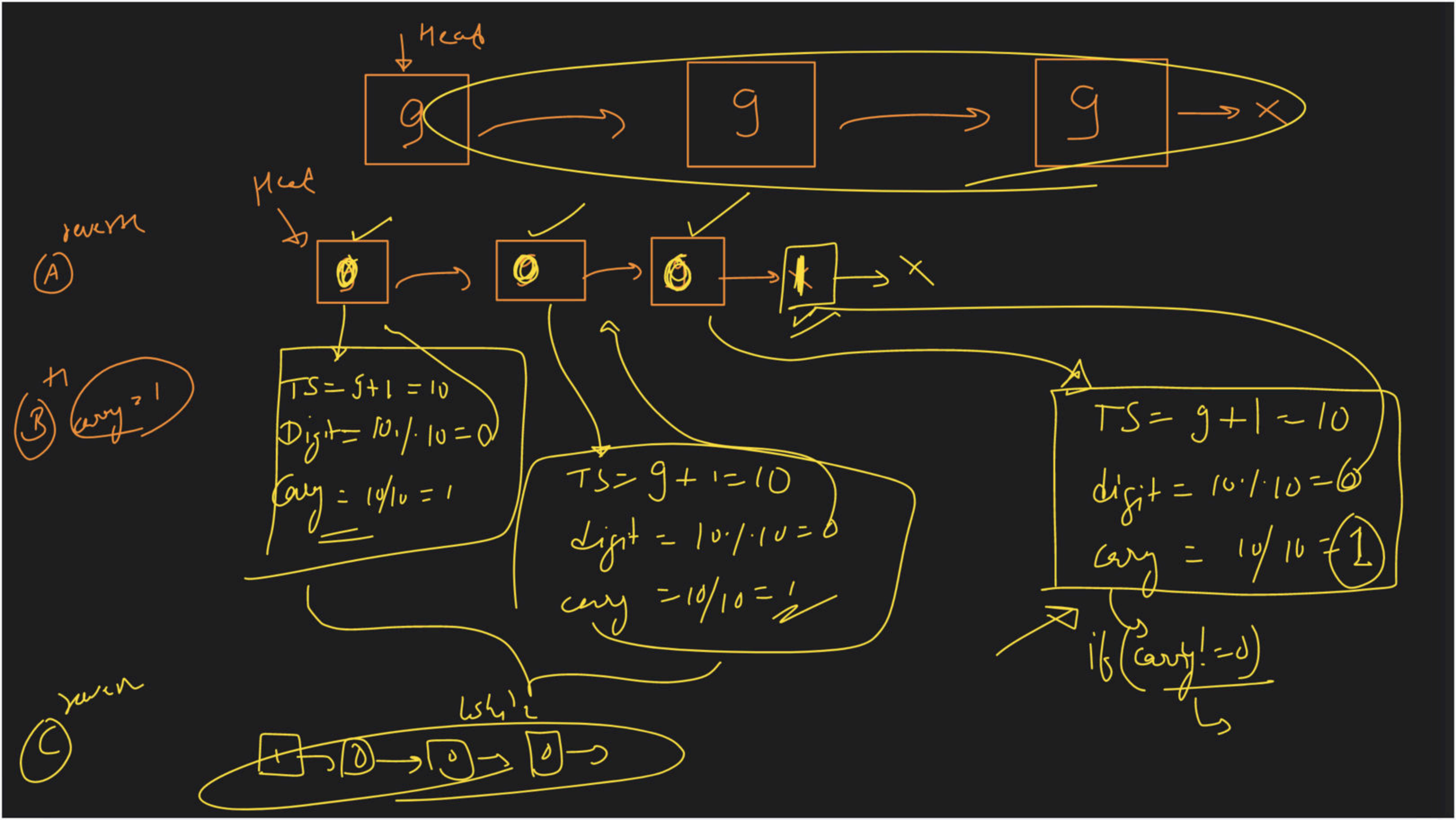


if curr != 0
& list khatam

curr Node

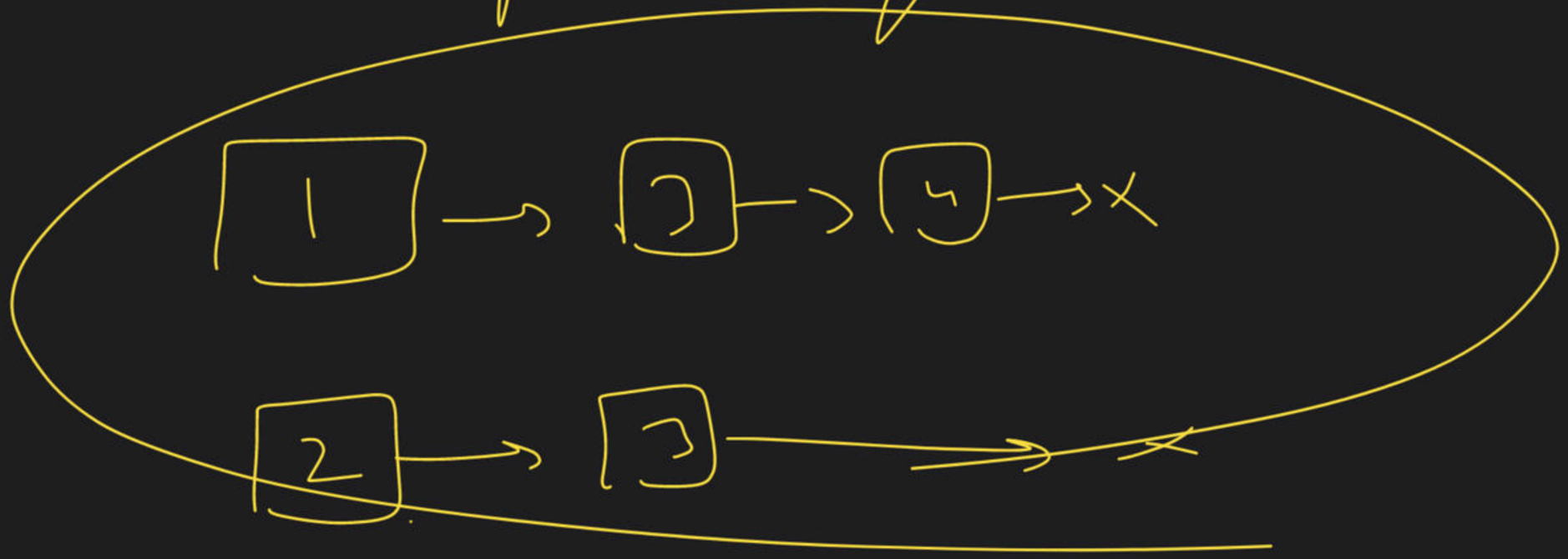

Head
i/p
1
3
8
X
Head
(1) recurse
9
3
1
X
(B) +1
carry = 1
totalSum = 8+1 = 9
digit = 9 % 10 = 9
carry = 9/10 = 0
3rd call
(C) return
Head
1
3
9
∧

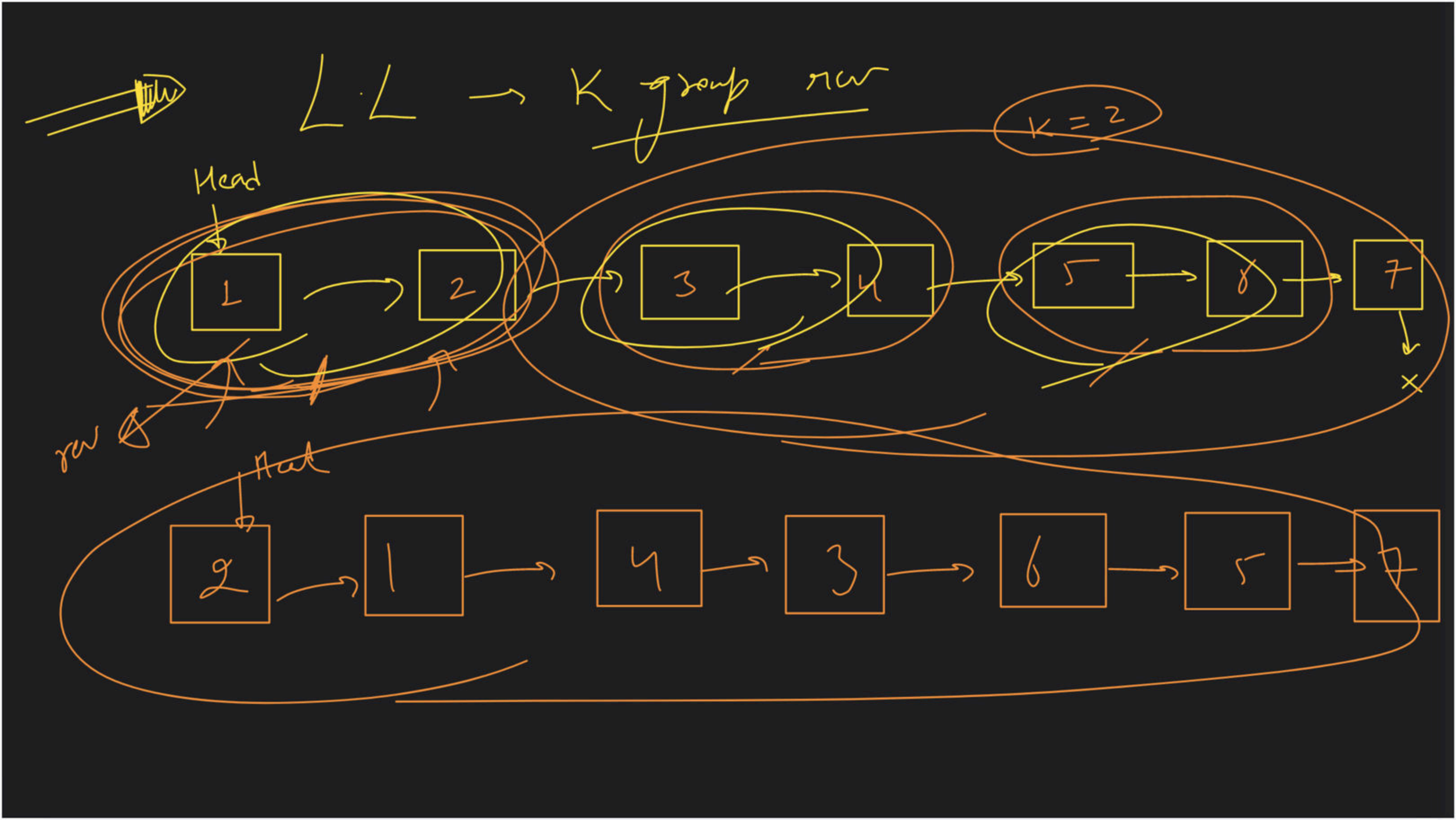

head
1
9
9
X
rev
head
0
0
9
alay sc
+1
2
carry = 1
totalSum = 9+1 = 10
digit = 10 % 10 = 0
carry = 10/10 = 1
totalSum = 9+1 = 10
digit = 10 % 10 = 0
carry = 10/10 = 1
totalSum = 1+1 = 2
digit = 2 % 10 = 2
carry = 2/10 = 0
while loop
rev
3
2
0
0

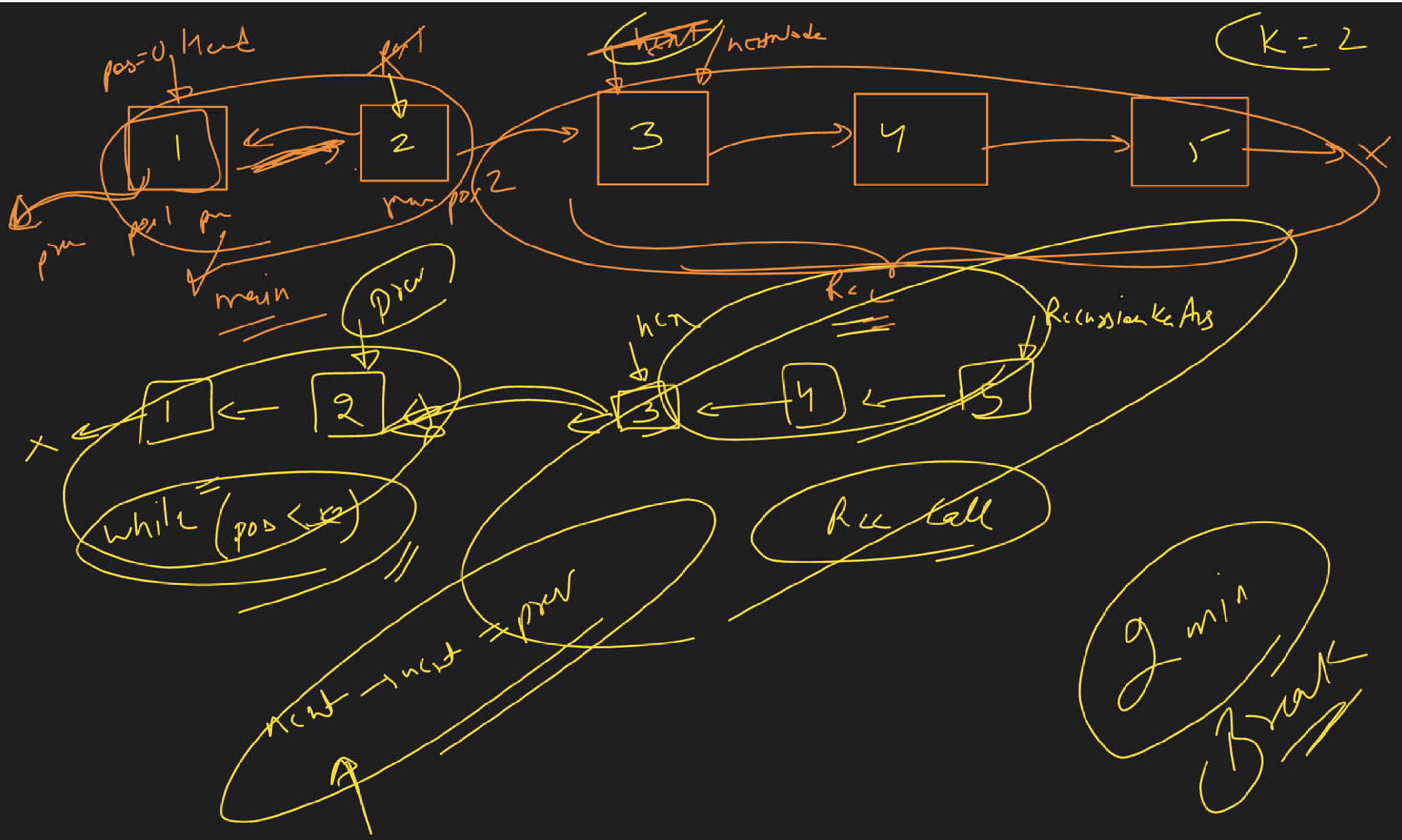

Head
9
9
9
X
Head
0
0
0
1
X
reverse
A
B
carry = 1
TS = 9+1 = 10
Digit = 10 % 10 = 0
carry = 10/10 = 1
TS = 9+1 = 10
digit = 10 % 10 = 0
carry = 10/10 = 1
TS = 9+1 = 10
digit = 10 % 10 = 0
carry = 10/10 = 1
if (carry != 0)
C
reverse
1
0
0
0

add 2 numbers

represented by Linked List




L.L → K group rev
K=2
Head
1
2
3
4
5
6
7
rev
Head
2
1
4
3
6
5
7



K = 2
pos=0, head
main
prev
while (pos < k)
Rec
Recursion ke Ans
Rec call
9 min
Break

(2)

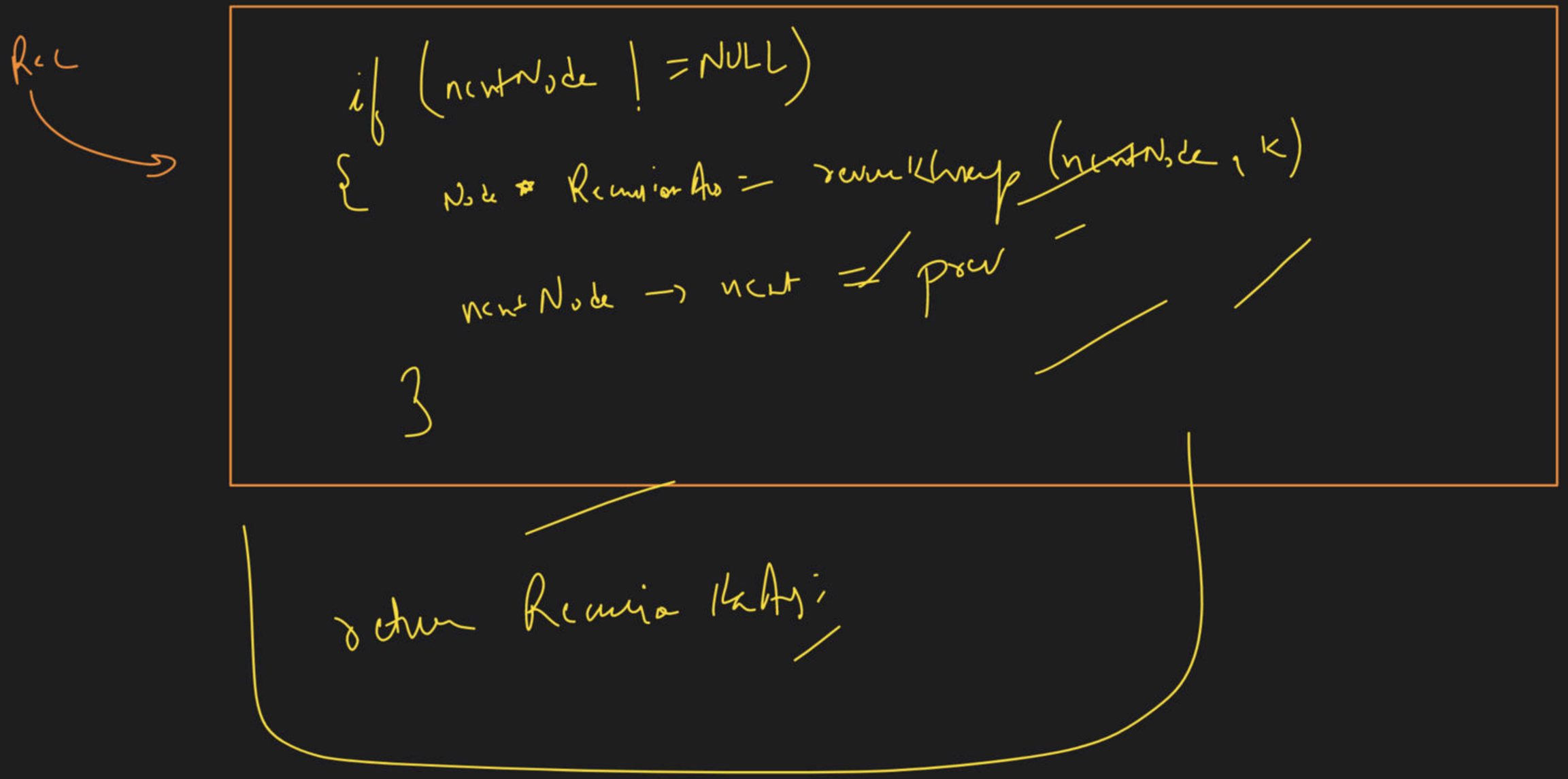
```
ListNode* reverseKGroup(ListNode* head, int k)
{
    if (head == NULL)
        return head;

    if (head->next == NULL)
        return head;

    // I can
    if (len >= k)
    Node* prev = NULL;
    while (pos < k)
    {
        pos++;
        nextNode = curr->next;
        curr->next = prev;
        prev = curr
        curr = nextNode;
    }
```

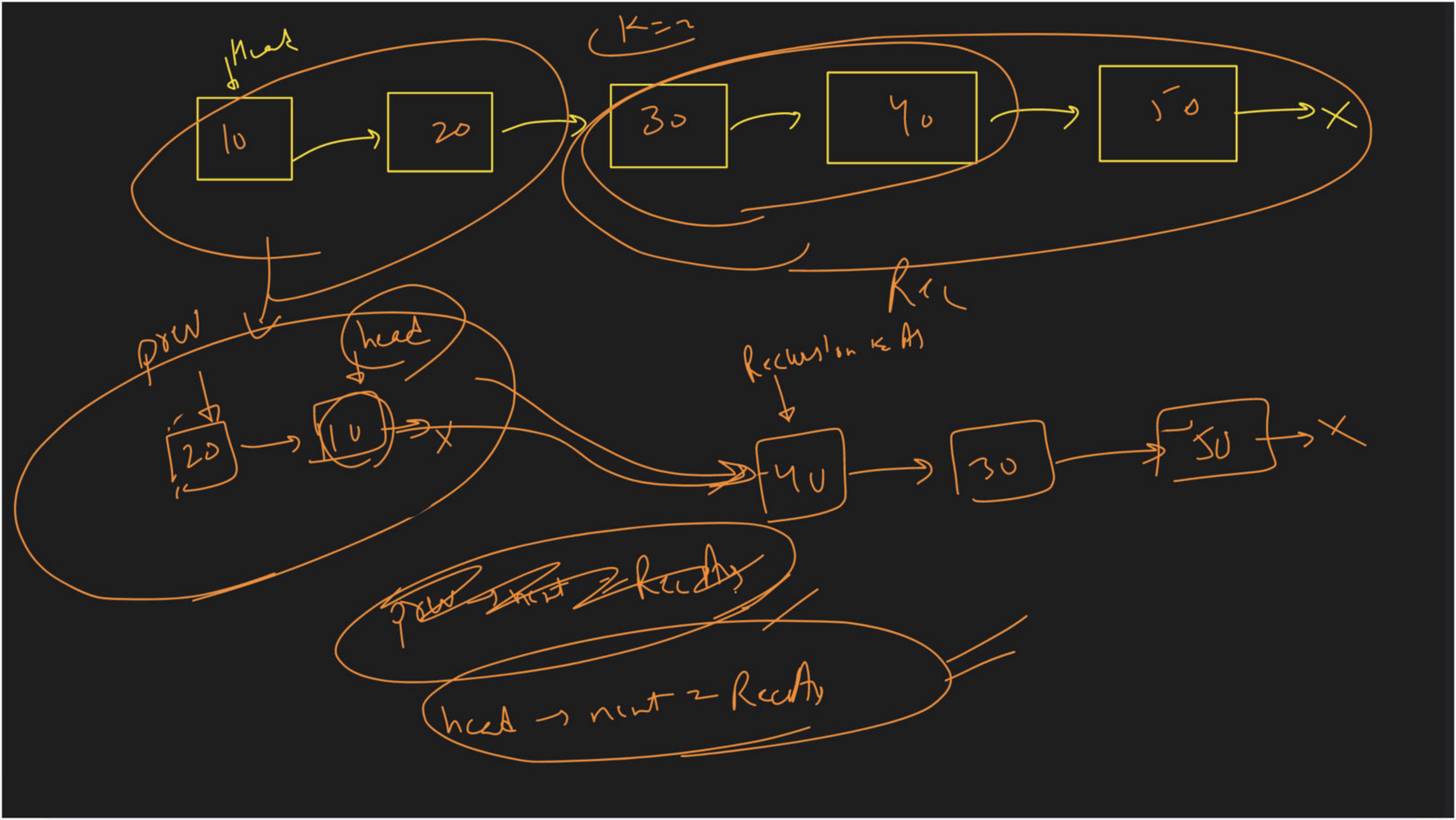
reverse first k node

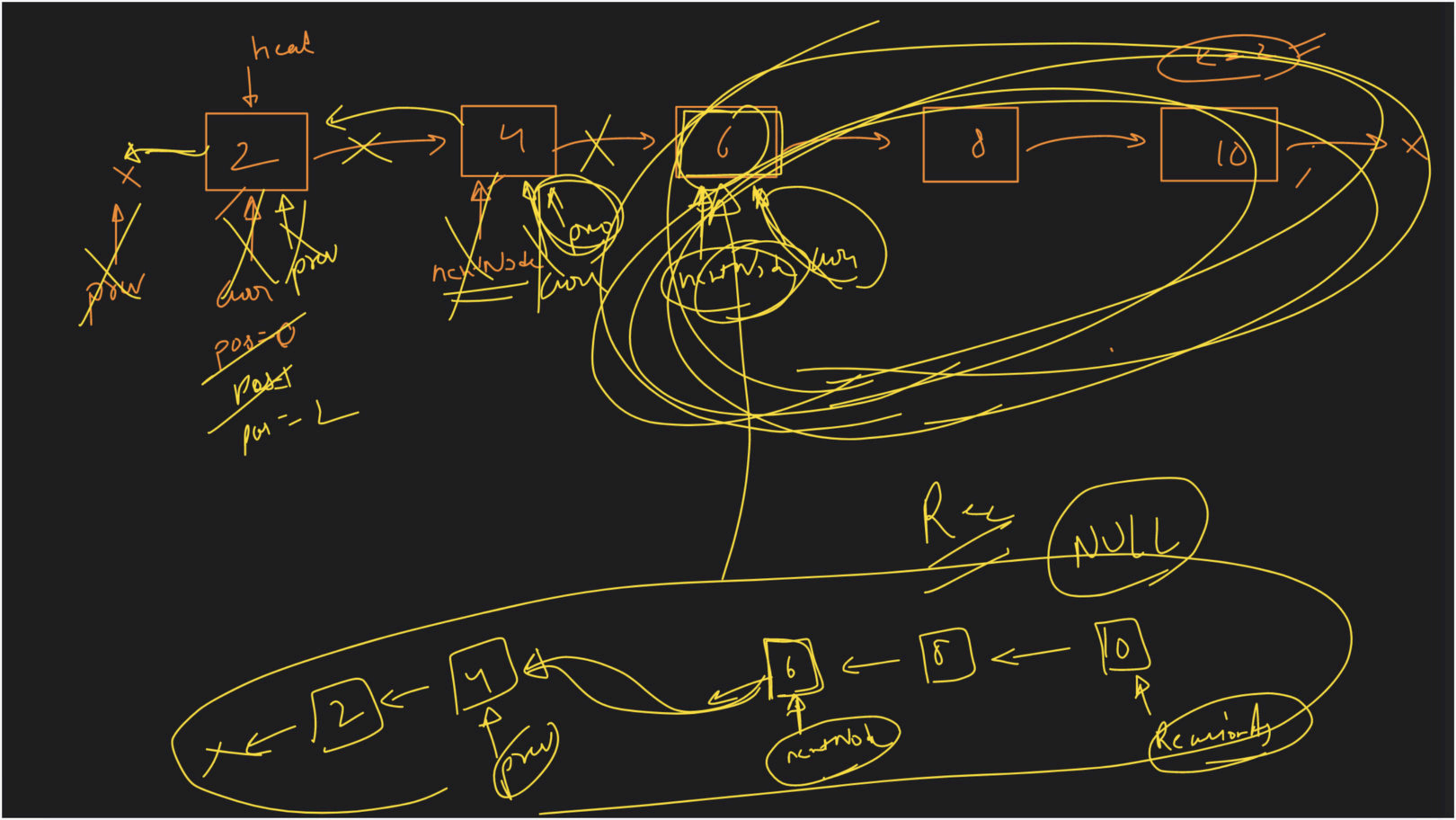
```
Node* next = curr->next;
Node* curr = head;
int pos = 0
```

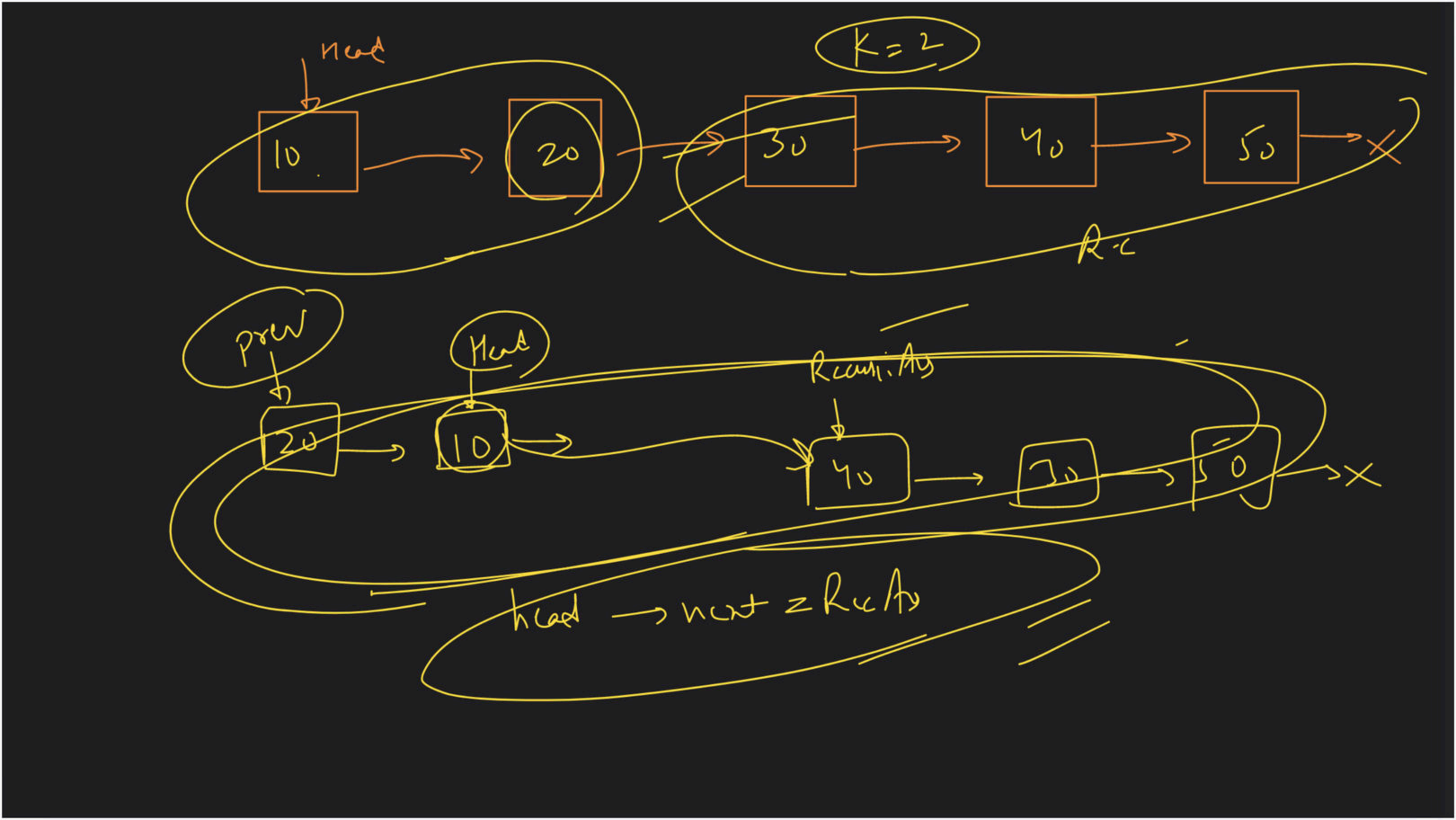
Leetcode 25

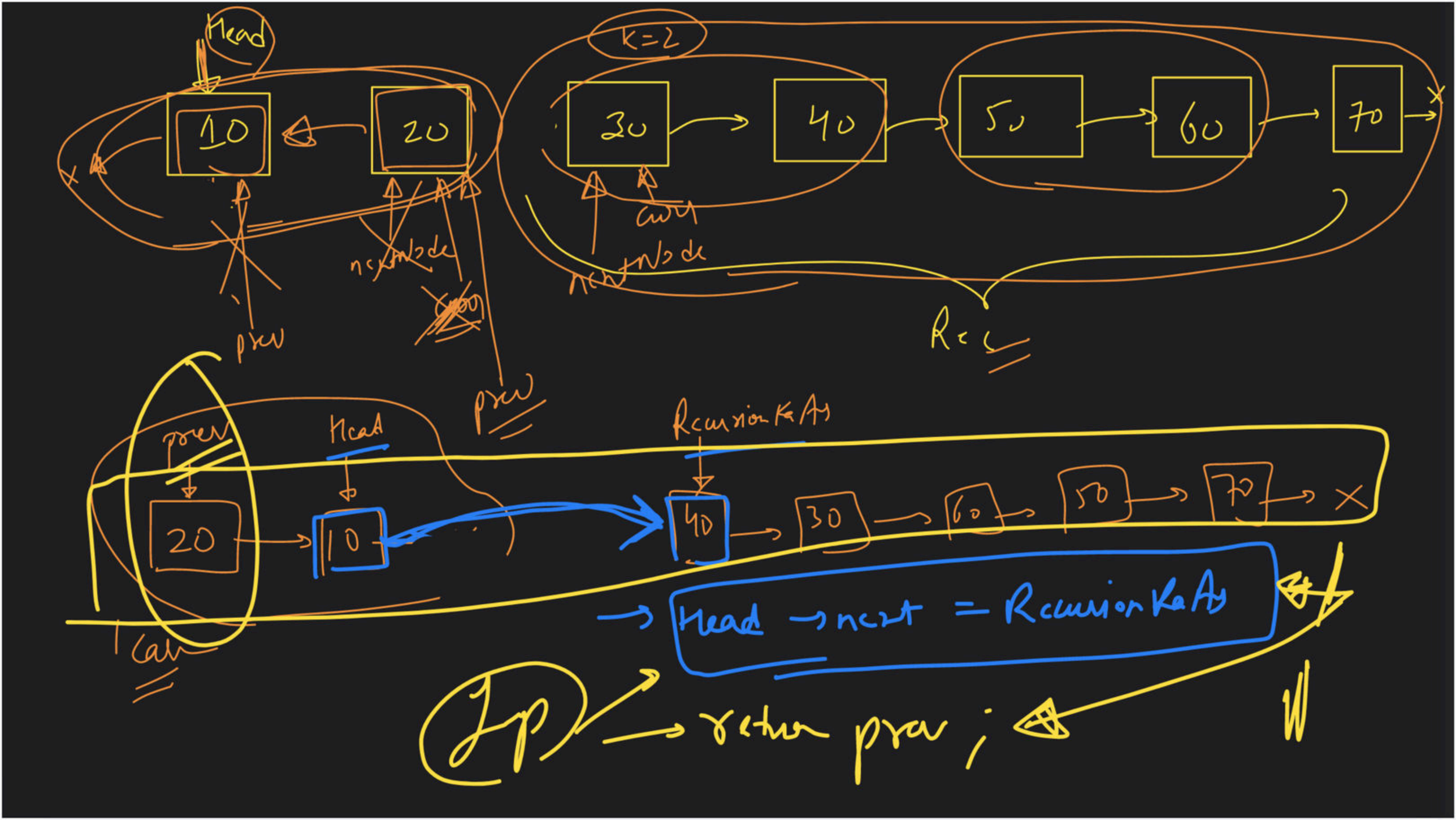
Linked List Reverse in K-groups

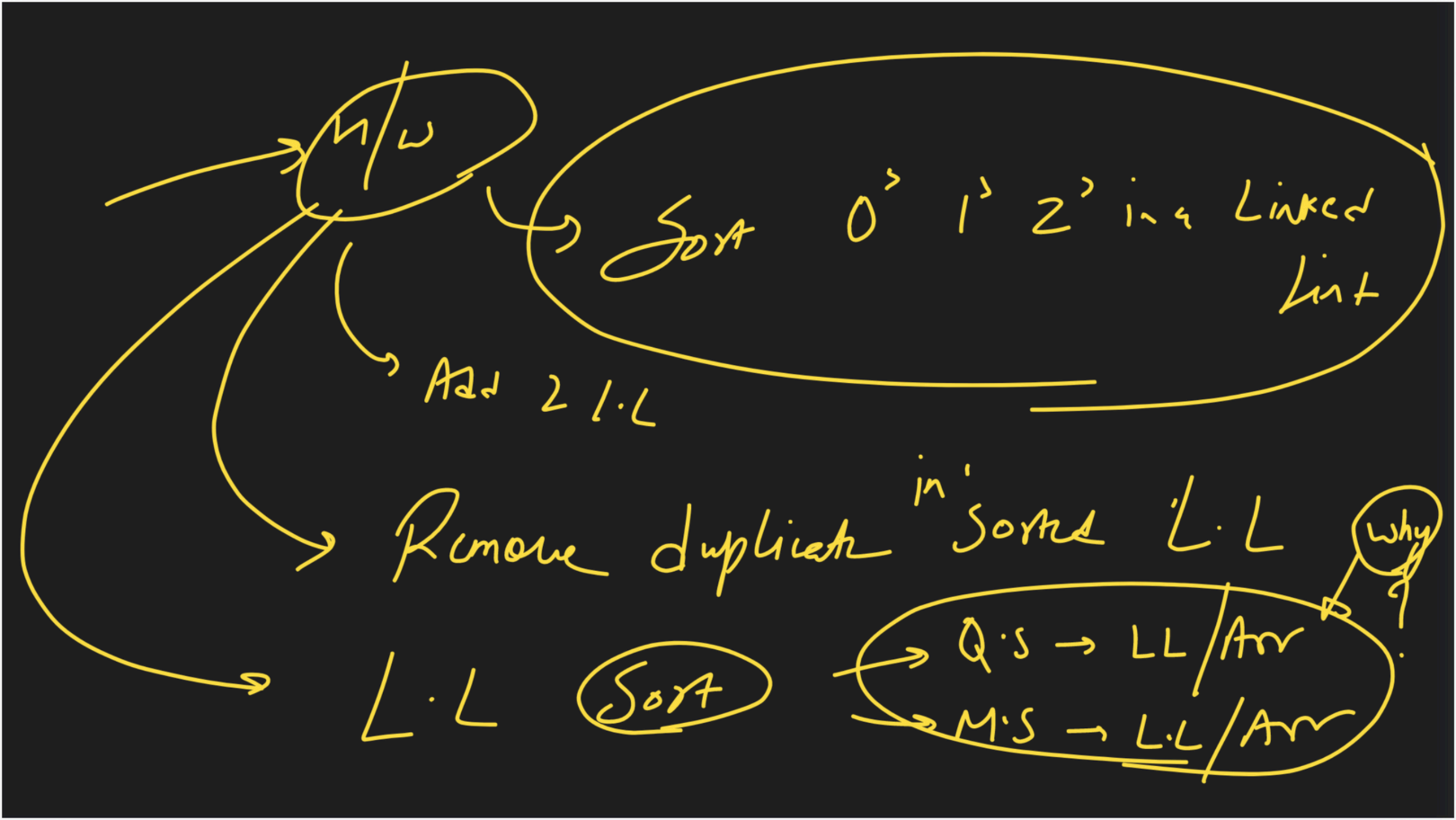

Rec
if (nextNode != NULL)
{
Node * RecursionAns = recursiveSearch(nextNode, k)
nextNode -> next = prev
}
return RecursionAns;


head
10
20
K=2
30
40
50
X
prev
head
20
10
X
Recursion ka A
40
30
50
X
prev -> next = Recall
head -> next = Recall

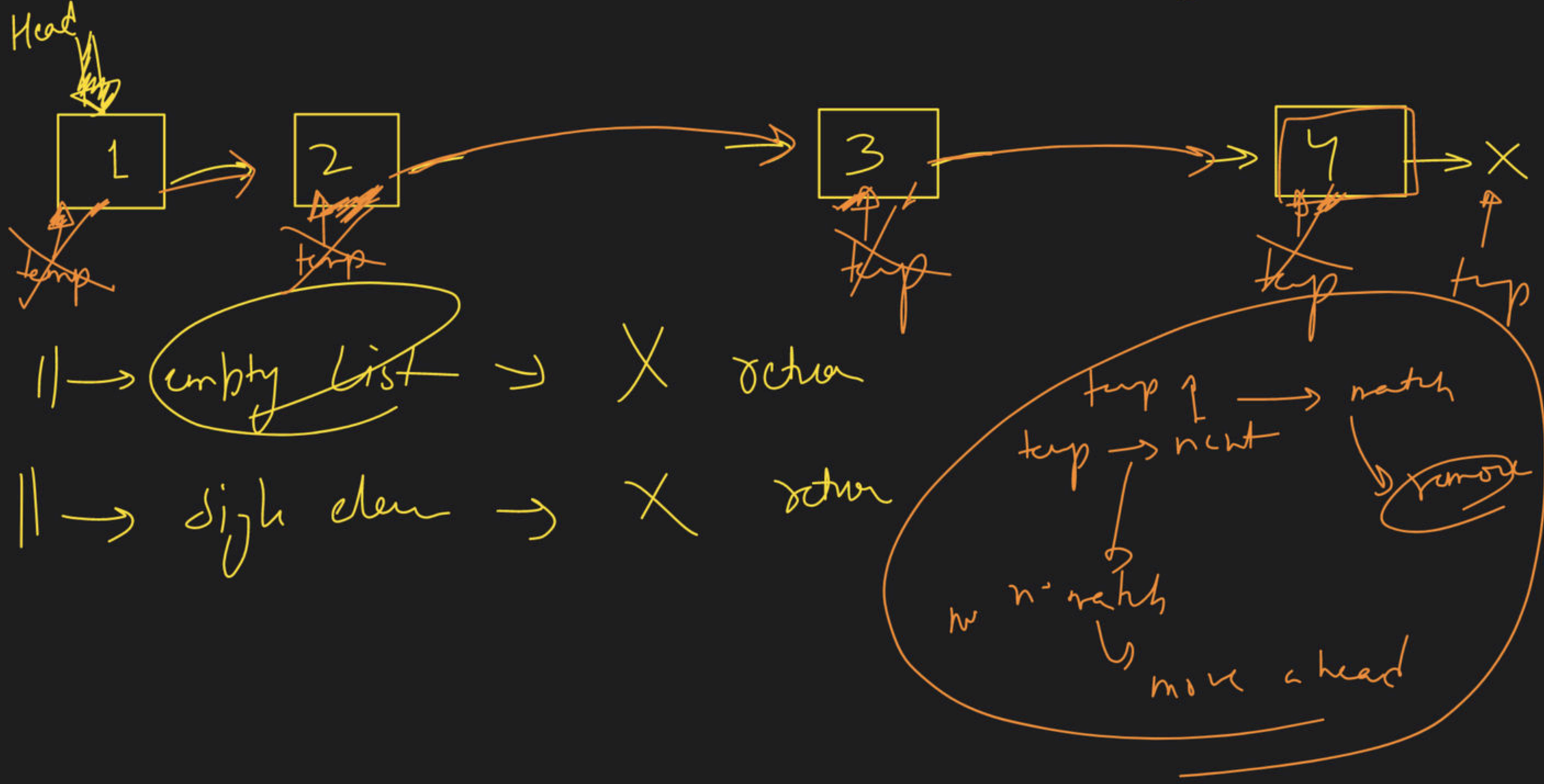

head
2
4
6
8
10
prev
curr
pos=0
pos=1
pos=2
NULL
Recursion


head
10
20
K = 2
30
40
50
X
R.C
prev
head
20
10
Recursive Ans
40
30
50
X
head → next = Rec Ans

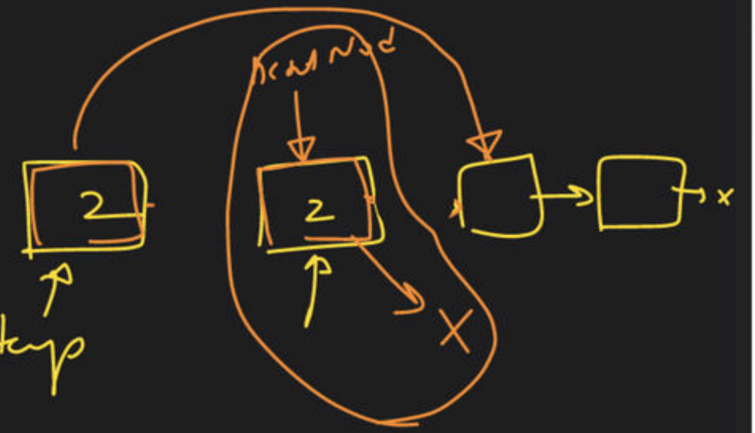

Head
10
20
30
40
50
60
70
k=2
prev
nextNode
curr
Recursion Ka Ans
Head → next = Recursion Ka Ans
return prev ;


H/W
Sort 0's 1's 2's in a Linked List
Add 2 L.L
Remove duplicate in Sorted L.L
Why
L.L
Sort
Q.S → LL / Arr
M.S → L.L / Arr

→ Remove duplicate in Sorted L.L



```
Node * Solve( Node * head )
{
    if (head == NULL)
        return head;

    if (head -> next == NULL)
        return head;

    Node * temp = head;
    while ( temp != NULL )
    {
        if (temp->next != NULL && temp -> data == temp->next->data)
        {
            //remove
        }
        else {
            temp = temp->next;
        }
    }
    return head;
}
```




```
//remove
Node * nextNode
    = temp->next
temp -> next = nextNode->next
nextNode -> next = NULL
delete nextNode
```